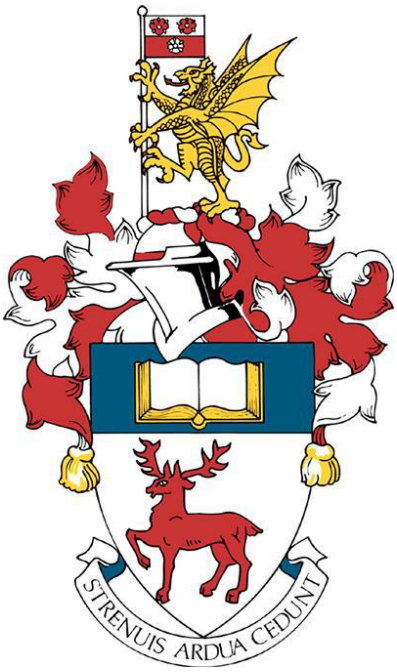


University of Southampton

FEEG3003 Individual Project

Optimal Design of PEM Electrolyser with Integration of Renewable Power Supply for Forecasting of Green Hydrogen Production using Machine Learning

Haseeb Mahmood

Supervisor: Dr Ranga Dinesh Kahanda Koralage

Date: 03/05/2024

Word count: 10318

”This report is submitted in partial fulfillment of the requirements for the Master of Mechanical Engineering, Faculty of Engineering and the Environment, University of Southampton”

Declaration

I, Haseeb Mahmood declare that this thesis and the work presented in it are my own and has been generated by me as the result of my own original research.

I confirm that:

1. This work was done wholly or mainly while in candidature for a degree at this University;
2. Where any part of this thesis has previously been submitted for any other qualification at this University or any other institution, this has been clearly stated;
3. Where I have consulted the published work of others, this is always clearly attributed;
4. Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work;
5. I have acknowledged all main sources of help;
6. Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself;
7. None of this work has been published before submission

Acknowledgements

I want to start by expressing my sincere thanks to my supervisor Dr Ranga Dinesh Kahanda Koralage, for his continuous guidance and support during my project. Your weekly meetings were instrumental in keeping me on track, and your insightful suggestions gave me the drive needed to complete this project.

I would also like to thank my family and friends for their support and encouragement during my project. At times when I was struggling or felt stuck, it was their motivation that helped me persevere. I am very privileged to have such a supportive circle.

Table of Contents

Declaration 1
Acknowledgements 2
Abbreviations 5
Abstract 6
1. Introduction 7
1.1 Green Hydrogen Production for a Hydrogen Economy 7
1.2 Hydrogen Production via Water Electrolysis 7
1.3 Proton Exchange Membrane Water Electrolysis 8
1.4 Renewable Energy and Water Electrolysis 9
1.5 Machine Learning Background 10
1.5.1 Machine Learning and Green Hydrogen Production 10
2. Literature review 12
2.1 Introduction 12
2.2 Prediction and Optimisation Using Machine Learning 12
2.3 Machine Learning for Predicting PEM Electrolyser Design 14
2.4 Machine Learning for Simulation of PEM Electrolysers 16
2.5 Simulation and Machine Learning-based Forecasting of Green Hydrogen Production 16
2.5.1 Wind-powered Hydrogen Production Simulation 17
2.5.2 Prediction of Environmental Parameters for Hydrogen Production Using Machine Learning 17
2.6 Current Gaps in the Literature for Optimising PEM Electrolysers and Simulating Green Hydrogen Production 17
2.7 Aims and Objectives 18
3. Methodology 20
3.1 Data Collection: 20
3.2 Data Pre-processing 21
3.3 Machine Learning Framework 25
3.3.1 Software 25
3.3.2 Machine Learning Models 25
3.3.3 Hyperparameter Tuning 26
3.4 Model Evaluation 26
3.5 Feature Importance 28
3.6 Optimisation of Numerical Parameters 28

3.7 Integration with Renewable Energy Source to Forecast Hydrogen Production 29
4. Results and Discussion 32
4.1 Correlation Matrix 32
4.2 Machine Learning Models Comparison 33
4.3 Feature Importance Comparison 36
4.3.1 Process Parameter Importance 40
4.3.2 Optimal Design 41
4.4 Optimal Process Parameters 43
4.5 Optimal Design and Comparison with Previous Study 45
4.6 Hydrogen Production Forecasting Based on Dynamic Wind Energy Inputs 47
5. Conclusion 50
6. Limitations and Future Directions 51
7. Bibliography 52
8. Appendices 56
8.1 Appendix A: Hydrogen Production Simulation Graphs 56
8.2 Appendix B: Python Code 58

Abbreviations

Abbreviation	Meaning
RF	Random Forest
DT	Decision Tree
DTR	Decision Tree Regressor
KNN	K Nearest Neighbour
XGB	Extreme Gradient Boosting
GBM	Gradient Boost Machine
SVM	Support Vector Machine
ANN	Artificial Neural Networks
RMSE	Root Mean Square Error
MAE	Mean Absolute Error
R2	Coefficient of Determination
AT	Anode Type
CT	Cathode Type
AGDE	Anode Gas Diffusion Electrode
CGDE	Cathode Gas Diffusion Electrode
GDE	Gas Diffusion Electrode
MT	Membrane Type
CC	Cathode Catalyst
AC	Anode Catalyst
CE	Catholyte
AE	Anolyte
ES	Electrode Shape
AFT	Anode Flow Type
CFT	Cathode Flow Type
DI	Deionised
PEM	Proton Exchange Membrane
PFSA	Polysulfanated
SCADA	Supervisory Control and Data Acquisition
EDA	Exploratory Data Analysis
DF	Dark Fermentation
PV	Photovoltaic
M	Molar
Pt/C	Platinum on Carbon

Abstract

The upcoming role of hydrogen is in its use as an energy carrier, and green hydrogen production is pivotal for advancing the hydrogen economy. Green hydrogen production is riddled with many issues, with prediction and optimisation of its related technologies suggested to be some of the most significant.

This report explores using machine learning (ML) tools to optimise the design and operational conditions of a proton exchange membrane (PEM) electrolyser. We applied different ML models to predict the hydrogen flow rate. We found that random forest (RF) was the most accurate model for predicting hydrogen production for PEM technologies based on model evaluation techniques such as mean absolute error (MAE), root mean square error (RMSE) and the coefficient of determination (R2). We explored the most influential process parameters based on a feature importance analysis. We optimised the design and operational conditions using the feature importance analysis and optimisation techniques for maximum hydrogen production. We also examined the effects of some variables on hydrogen production with partial dependence plots. We also present the coupling of a renewable energy source for simulation and forecasting green hydrogen production. In summary, our study shows the potential ML has in addressing issues related to green hydrogen production.

1. Introduction

1.1 Green Hydrogen Production for a Hydrogen Economy

The demand for energy is ever-increasing, and by 2050, the amount of energy required worldwide is expected to increase from 13.6 billion tons of oil equivalent in 2010 to 44.6 billion tonnes of oil equivalent [2]. Green hydrogen production is pivotal for advancing the hydrogen economy in this context. Green hydrogen production refers to producing hydrogen from purely renewable energy sources like solar and wind power, emitting low to no greenhouse gases. The significant and upcoming role of hydrogen seems to be in its use as an energy carrier, and it has the highest energy content per weight of any fuel, approximately 120 MJ/Kg [3]. When hydrogen is burned, it produces water as its only by-product, making it a zero-emission fuel when produced from purely renewable methods. Despite the benefits of green hydrogen production, it is currently costlier than other hydrogen production methods, with costs ranging from €3 to €8 per kg, compared to €1 to €2 per kilogram for grey hydrogen production. Although in some regions, green hydrogen production costs are projected to decrease to approximately €1 to €1.5 per kg by 2050 [4] . With its potential to decarbonise various sectors and contribute to the hydrogen economy, green hydrogen production can significantly fulfil growing global energy needs and combat climate change [5].

1.2 Hydrogen Production via Water Electrolysis

There are many different methods by which green hydrogen can be produced, such as Dark fermentation (DF), in which anaerobic bacteria are used to break down feedstock using enzymes, producing hydrogen in the process [6]. However, the most common method is water splitting using electrolysis, where the electricity that powers this process comes from renewable sources, the overall equation for which is shown below :

$$2H_2O(l) \rightarrow 2H_2(g) + O_2(g)$$

Equation 1 - Overall water electrolysis equation

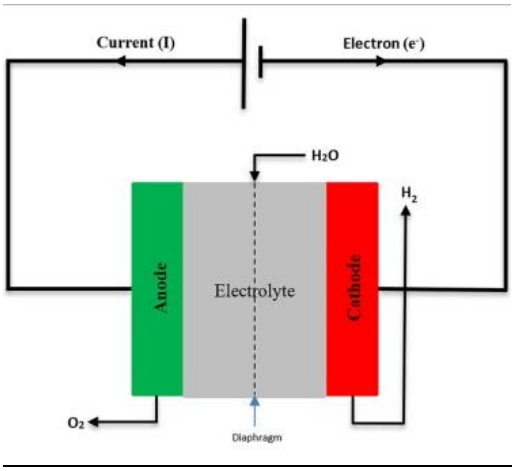


Figure 1 - Basic electrolyser diagram [7]

In electrolysis, electricity is used to break down a compound into its elements by passing an electric current through 2 electrodes in its fluid form [8]. Figure 1 shows a basic electrolyser diagram. The anode is the electrode where the oxidation (loss of electrons) reaction occurs, and oxygen is produced. The electrons then travel through the circuit to the cathode, the electrode where the reduction (gain of electrons) reaction occurs, and hydrogen is produced. The electrolyte refers to the substance which provides the ionic conductivity needed for electrolysis. In some electrolyser technologies, a separator or membrane is used, which is a component that separates the electrodes whilst still allowing ions to pass through, preventing the gases produced at each electrode from mixing.

There are four main types of electrolyser technologies for producing hydrogen: Alkaline and Polymer exchange membranes (PEM), which are already at a commercial level, whilst anion exchange membrane and solid oxide electrolysers are still in their research and development phase [9] [10]. The main difference between these different electrolyser technologies is their electrolyte type. However, research shows that PEM electrolysis is the most widely used water electrolysis technology and one of the most promising for scaling up green hydrogen production [2].

1.3 Proton Exchange Membrane Water Electrolysis

Proton exchange membrane (PEM) electrolysers operate on principles like PEM fuel cells. They use solid polysulfonated (PFSA) membranes as electrolytes, known for their exceptional proton conductivity and chemical stability [9]. PEM electrolysers stand out for several reasons: they can

achieve very high current densities, up to 10 A/cm², and produce hydrogen with a purity of up to 99.9999% [7]. Additionally, they operate at a low-temperature range of 30-80 °C, have high output pressures, and allow for a compact system design [9]. One of the most significant advantages of PEM electrolysers is their fast response time [9] [11] [7]. They can adjust the hydrogen production rate rapidly in response to changes in electrical power input. This is particularly beneficial in green hydrogen production, given the variable nature of renewable energy sources such as solar and wind energy, whose output fluctuates depending on weather conditions and time of day.

1.4 Renewable Energy and Water Electrolysis

Photovoltaic (PV) panels and wind turbines are the two most common renewable energy sources. PV systems consist of PV cells, which absorb incident solar radiation and convert it to electricity. Wind turbines use wind energy to spin turbines around a rotor, which then spins a generator, producing electricity. PV panels and wind turbines rely heavily on weather conditions and time of day; thus, they are intermittent power sources. For sporadic sources of power, there are various ways they can be integrated with electrolysers, which can be classified based on their connection with the grid:

On-grid systems – In this system, the grid powers the electrolysers, and the renewable energy source feeds energy back to the grid. The advantage of this is that the electrolysers can operate at full capacity without fluctuation due to renewable energy sources. For the hydrogen production process to remain ‘green’, the energy produced by the renewable source fed back to the grid must match the energy used by the electrolysers. [12] [13]

Off-grid systems – Off-grid systems refer to renewable energy sources powering electrolysers without support from the grid. Usually, these systems are coupled with energy storage devices, which are used to ease power fluctuations due to the intermittent nature of these renewable energy supplies [13] [12] [14]. However, direct coupling could also happen depending on the type of electrolyser technology, which refers to a renewable energy source powering the electrolyser without any intermediate energy storage device. For instance, the fast response times of PEM electrolysers make them ideal for the variable nature of these intermittent supplies.

Whilst integrating renewable energy with hydrogen production is promising, many projects are still in their pilot phase. For example, the Brande hydrogen pilot project explores an off-grid configuration, coupling an existing 3 MW onshore wind turbine to an electrolyser system to explore the efficiency of the electrolyser system over time [15].

1.5 Machine Learning Background

Machine learning (ML) is a branch of artificial intelligence that allows algorithms to learn and adapt by analysing large volumes of data to identify patterns and make decisions without human interaction [16].

There are a few categories that different ML models fall into:

1. **Supervised Machine learning** – This approach uses labelled data to train algorithms to learn the relationships between input and output variables to generalise from the training data so that the model can accurately predict outputs for unseen data.[16]
2. **Unsupervised Machine Learning –** Unsupervised machine learning is the opposite of supervised learning because the algorithms learn patterns from unlabeled data instead of labelled data. The algorithms essentially explore the data structure to find and pick up hidden patterns and groupings. [16]
3. **Ensemble methods** – Ensemble methods in machine learning are algorithms which combine the prediction from multiple base algorithms (known as weak learners) to improve accuracy. For example, random forest (RF) is a machine learning ensemble algorithm that combines the outcomes of multiple decision tree (DT) algorithms.[16]

1.5.1 Machine Learning and Green Hydrogen Production

ML is used in various applications, and integrating machine learning and AI tools with green hydrogen production could help address significant issues related to green hydrogen production. For example, the optimal design of PEM electrolysers requires the maximisation of the hydrogen production rate, and understanding the influence various parameters, such as gas diffusion electrodes, catalysts, membranes, etc, have on electrolyser performance is crucial for this [1] [17]. Machine learning can simulate and optimise electrolyser design without the time and financial impacts of traditional experimental methods and explore the relationship of various other process parameters on hydrogen production.

It is suggested that some of the most significant scale-up issues associated with green hydrogen production will be optimal design, prediction and process optimisation, which ML can be used to help address [1] [18].

2. Literature review

2.1 Introduction

This literature review will explore the current literature on the role of ML tools in addressing issues related to green hydrogen production and its related technologies. We examine various studies that have used ML in this context, highlighting their methodologies and findings. As this is a developing area, there are, however, few studies which have explored the use of ML techniques for addressing challenges related to green hydrogen production. However, ML does show great potential for improving the hydrogen production process and, therefore, could significantly aid in advancing the hydrogen economy.

2.2 Prediction and Optimisation Using Machine Learning

A study by Mohammed Mahbub Kabir et al. presented a comprehensive application of different ML models for optimising and predicting green hydrogen production through DF and PEM water electrolysis technologies. The study compared different ML models' efficacy in predicting hydrogen production for these technologies. The ML models used were Random Forest (RF), Support Vector Machine (SVM), Gradient Boosting Machine (GBM), and Decision Trees (DT) to predict and optimise hydrogen production. The methodology presented was comprehensive, in which many data points were extracted through a bibliographic analysis. 1699 data points were used for Dark Fermentation, and 2007 data points were used for PEM electrolysis. The study evaluated the performance of all of the models using performance metrics such as the coefficient of determination (R2), root mean square error (RMSE), and mean absolute error (MAE). RF showed the best performance when forecasting hydrogen yield for PEM electrolysis, with the lowest errors and the highest R2 score out of all the models used, highlighting random forest’s ability to handle complex nonlinear relationships between different variables.

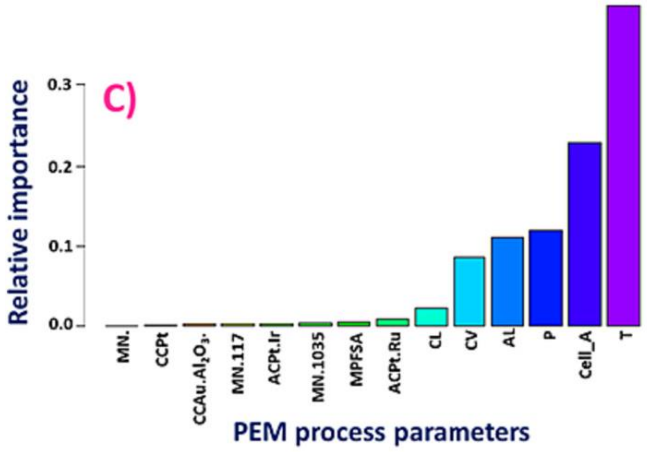


Figure 2 – RF feature importance [18]

The figure above shows the feature importance graph for the random forest model created through a permutation variable index analysis. The study identified that the critical process parameters for PEM electrolysis were temperature (with the highest relative importance), cell area, pressure, and voltage.

Through a partial dependency analysis, a tool used to visualise the effect of the predictors on the target variable, insights were provided on how specific parameters impact hydrogen production through PEM electrolysis. For example, the partial dependence analysis revealed an optimal temperature range and found that temperatures up to 35 degrees were ideal for maximising hydrogen production. Beyond this temperature, the efficiency of hydrogen production starts to decline. This range ensures the electrolysis process operates within a thermodynamically favourable window, helping to enhance electrolysis kinetics whilst maintaining the stability and durability of the electrolysis components. The findings provide insights into optimal operational conditions, which can be used to help refine electrolysis processes. [18]

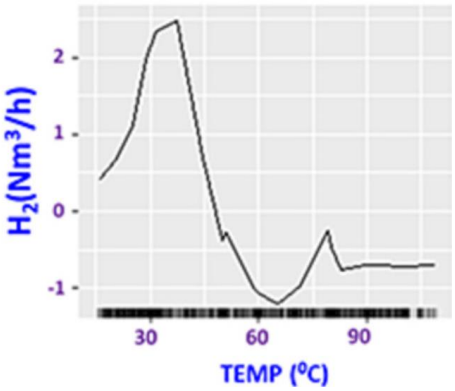


Figure 3 - Partial dependence plot of temperature on hydrogen production rate. [18].

2.3 Machine Learning for Predicting PEM Electrolyser Design

As of writing this report, there appear only to be two studies in the available literature that have used machine learning to predict electrolyser design parameters. Mohamed et al. successfully utilised polynomial regression and logistic regression models to predict a set of 11 electrolyser cell parameters from 4 input parameters, which were as follows: hydrogen production rate, cathode area, anode area, and cell design type (single or bipolar). The study found that the algorithms exhibited an average accuracy of 83.6% and MAE of 6.825, demonstrating high accuracy and robust performance [17]. However, a more comprehensive study was employed by Rui Yang et Al.

Rui Yang et al., employed a sophisticated ML framework, using K-nearest neighbours (KNN) and decision tree regression (DTR) models to predict cell design parameters for PEM electrolysers. The models were trained on an extensive dataset of 1062 design points derived from a comprehensive literature review and experimental setups. The study aimed to develop a model to predict 17 electrolyser design parameters from 5 input parameters: hydrogen production rate, electrode area, anode and cathode flow areas, and cell design type. The KNN algorithm was used for the classification of categorical variables related to the cell design of the electrolyser, such as current collector type, gas diffusion electrode, electrolyte type, and flow pattern shape, whilst the DTR algorithm was used for the regression problem, to predict continuous variables, such as voltage, current density, temperature, pressure and number of cells. A test set of the data not seen during the model's training was used to validate the model, utilising cross-validation techniques to evaluate the model's performance. The model was also validated using six

experimental setups, and the models showed remarkable accuracy, with a mean square error of 0.31. The low errors of the ML algorithms, when compared against experimental results, highlight the efficacy of the models in this context [1].

In addition to this, the study also successfully predicted the optimal parameters for a larger-scale electrolyser design of 3000 ml/min, as shown below:

Anode Type	Titanium
Cathode Type	Titanium
Anode GDE (Gas diffusion electrode)	Titanium mesh
Cathode GDE (Gas diffusion electrode)	Porous Titanium
Membrane Type	Nafion 110
Cathode catalyst	0.4mg Pt/cm^2
Anode catalyst	2 mg of Pt/cm^2 and 2 mg of IrO_2/cm^2
Catholyte	Deionised water
Anolyte	Deionised water
Anode flow type	
Cathode flow type	
Cell design number	10
Cell temperature (K)	333.15
Cell voltage (V)	17.11
Cell current density (A/cm^2)	0.87
Electrode area (cm^2)	100
Anode/cathode flow area (cm^2)	100
Pressure (ATM)	1
Hydrogen flow rate (ml/min)	3000

Table 1 - Input/predicted parameters for a larger-scaled PEM electrolyser design. [1]

These studies show the potential of ML in predicting and enhancing the design of PEM electrolysers, optimising design parameters for differing scales. The success of predicting parameters for a larger scale system of 3000ml/min also shows the scalability of ML applications in this field. Whilst the application of 2 models provided significant insights, the studies would

benefit from an additional comparative analysis between alternative models, evaluating the performance of alternative models to see which model is best suited for this application.

2.4 Machine Learning for Simulation of PEM Electrolysers

Mohammed et al. used ML to predict hydrogen production rate and cell current density for a PEM electrolyser using 15 input parameters, such as anode type, cathode type, electrode area, anode flow area, power, cell temperature, etc. The study used a dataset of 1203 data points, 1086 of which were used for model training and 117 for testing. The methodology of the study involved training and testing five different ML models: Artificial neural networks (ANN), Polynomial Regression, Support vector regression, K nearest neighbours (KNN) and decision tree regression (DTR). The models were trained using the training set and then evaluated using the test set, with MAE as the primary evaluation metric. Among these models, ANN emerged as the best and most effective model for predicting hydrogen production rate and current density. This is due to the model's ability to handle complex non-linear relationships, especially when considering how a PEM electrolyser operation involves many interactions between operating parameters, such as temperature, pressure, electrode and membrane types. ANN can accurately capture the relationship between these variables, especially in this context.

A further element of the study was the development of a graphical user interface for simulating electrolyser hydrogen production. Creating a graphical user interface allows a broader range of users to easily set up simulations using the models and parameters tested in the study, bridging the gap between research and practical application. [19]

2.5 Simulation and Machine Learning-based Forecasting of Green Hydrogen Production

Green hydrogen production relies heavily on renewable energy sources, and simulation and forecasting can be used to integrate these intermittent and variable energy supplies more effectively. For example, by understanding when and how much renewable energy will be available, the amount of hydrogen that can be produced can be forecasted for different periods.

2.5.1 Wind-powered Hydrogen Production Simulation

Torbjørn Egeland-Eriksen et al. simulated an offshore hydrogen production plant using actual data from a 2.3 MW offshore wind turbine, which included wind power production and wind speed, across five 31-day periods. The study used this dataset as an input to an electrolyser model, which was developed and implemented in Matlab/Simulink and successfully forecasted 17,242 kg over a 31-day period with a 1.852 MW electrolyser. Including electricity price data from the Nord pool market, where the wind turbine was located, allowed for a comprehensive cost analysis to calculate the cost of hydrogen production. It found that the lowest production cost was $4.53 per kg of hydrogen, demonstrating wind-powered hydrogen production's economic feasibility. [14]

2.5.2 Prediction of Environmental Parameters for Hydrogen Production Using Machine Learning

ML has also been used to forecast hydrogen production by predicting environmental parameters. Amel Ali Alhussan et al used a Recurrent Neural Network model, trained using a solar irradiance dataset to forecast the level of solar energy for subsequent periods. They then coupled the model with mathematical models for the PV and PEM electrolyser systems to forecast hydrogen production. The study forecasted an average of 0.622 kg/day for the subsequent season. [20]

Similarly, Syed Altan Haider et al. used various ML models to forecast Islamabad's global horizontal irradiance. Their work involved training and evaluating different ML models using historical weather data collected over 13 months. Upon training, the models were then used to forecast the global horizontal irradiance for 45 days. Coupling the forecasts with a PV electrolytic mathematical model allowed for the prediction of hydrogen. Using the prophet machine learning model, which exhibited a mean absolute percentage error of 3.7%, the study forecasted that Islamabad's average daily hydrogen production is 99300 kg/km^2. [21]

Forecasting hydrogen production in specific areas can help assess whether a particular location is suitable for a green hydrogen production plant and can aid in the planning and development of renewable energy projects.

2.6 Current Gaps in the Literature for Optimising PEM Electrolysers and Simulating Green Hydrogen Production

Although significant progress has been made in using ML to enhance green hydrogen production technologies, there are significant research gaps which remain that could further use the full

potential of this technology. Firstly, whilst existing studies have explored the relationship some parameters have on hydrogen production rates, there is a significant gap in using ML models in conjunction with optimisation techniques for the selection of the most optimal parameters to maximise hydrogen production rates, helping to enhance the efficiency of PEM electrolysers through optimal design and process optimisation.

Furthermore, another significant gap involves using these optimised electrolyser ML models with real-world data from renewable energy sources, such as solar and wind energy. This would allow for the simulation of green hydrogen production using ML and forecasting the amount of hydrogen that could be produced based on varying renewable energy inputs. This would also help us assess the optimised electrolyser's performance under real-world operating conditions.

2.7 Aims and Objectives

The main goal of this study is to show the potential of ML techniques in optimising PEM electrolyser design and integrating this with renewable energy for green hydrogen production. The specific aims and objectives are as follows:

1. Comparative analysis of machine learning models for predicting hydrogen production

- **Objective:** To evaluate and compare various machine learning models to determine which model best predicts hydrogen flow rate in PEM electrolysers. This will involve comparing performance metrics of different models, such as mean absolute error (MAE), root mean square error (RMSE), and the coefficient of determination (R2).

2. Optimisation of PEM electrolyser design:

- **Objective:** Using machine learning to optimise operational parameters of PEM electrolyser for maximum hydrogen production, which includes:
    - Identifying the optimal material configuration and numerical parameters, such as temperature pressure and electrode materials, for maximum hydrogen flow rate.
    - Analysing the relationship numerical parameters have on the hydrogen flow rate and each other
    - Identify which features most significantly affect the hydrogen flow rate.

3. Integration with renewable energy sources:

- **Objective:** To develop a model which integrates the optimised PEM electrolyser configuration with dynamic power inputs from a renewable energy source
    - The model will forecast the hydrogen production based on varying energy inputs, simulating the performance of the electrolyser under actual world operating conditions.
    - The main aim is to show that ML can also be used for simulation and prediction instead of mathematical modelling or alternate computational modelling techniques.

3. Methodology

3.1 Data Collection:

The data availability for PEM electrolysis technologies is extremely limited. For our modelling, a dataset from a previous study was used, in which 2 ML algorithms were used to predict electrolyser configurations based on various input parameters (study covered in section 3.2). The dataset contains a variety of numerical and categorical parameters for PEM electrolysis, as shown below.

Numerical Variables		
Numerical columns	**Min value**	**Max value**
Electrode area (cm^2)	9	100
Anode/Cathode flow area (cm^2)	4	50
Cell design number	1	10
Cell voltage (V)	1.38	19
Cell current density (A/cm^2)	0.000241	2
Power (W)	0.002222	1280
Power (W/cm^2)	0.000361	25.6
Water flow rate (ml/min)	1	1000
Hydrogen flow rate (ml/min)	0	5000
Cell temperature (K)	293.15	403.15
Pressure (ATM)	1	3

Table 2 - Overview of numerical variables [1]

Categorical Variables	
Anode/Cathode Type (AT/CT)	Carbon plate, Ni foam, Titanium, 316 stainless steel, Polypropylene plate coated with silver
Anode/Cathode Gas Diffusion Electrode (AGDE/ CGDE)	Porous carbon paper, Nickel (Ni) foam, Titanium, Carbon cloth, Titanium mesh, Carbon paper, Titanium felt, Stainless steel double felt layer
Membrane Type (MT)	Nafion117, Nafion115, Nafion110, Solvay Solexis
Cathode Catalyst (CC)	Molybdenum disulfide (MoS_2), Bio-engineered d hexagon-shape d Co3O4 nanoplates on Herring deoxyribonucleic acid (DNA), Platinum on carbon (Pt/C), 40% Pt/C, Platinum (Pt), 0.5 mg of Pt/cm^2, 0.4 mg of Pt/cm^2
Anode Catalyst (AC)	MoS2, Tungsten diselenide (WSe_2), Ruthenium (RuO_2) or iridium oxide (IrO_2), 1.5 mg of IrO_2/cm^2, 2 mg of Pt/cm^2 and 2 mg of $IrO2/cm^2$, IrO_2

Anolyte/Catholyte (AE/CE)	Deionised (DI) water, 0.5 M (molar) of choline chloride aqueous solution, 0.1 M of H_2SO_4, Steam water
Cell design type	Single or Bipolar
Electrode shape (ES)	Rectangle, Rectangular, Round
Anode/Cathode Flow type (AFT/CFT)	1 - 29

Table 3 - Overview of categorical variables [1]

The dataset also included various flow pattern shapes, which were numbered for classification as 1 – 29, as shown in Figure 4:

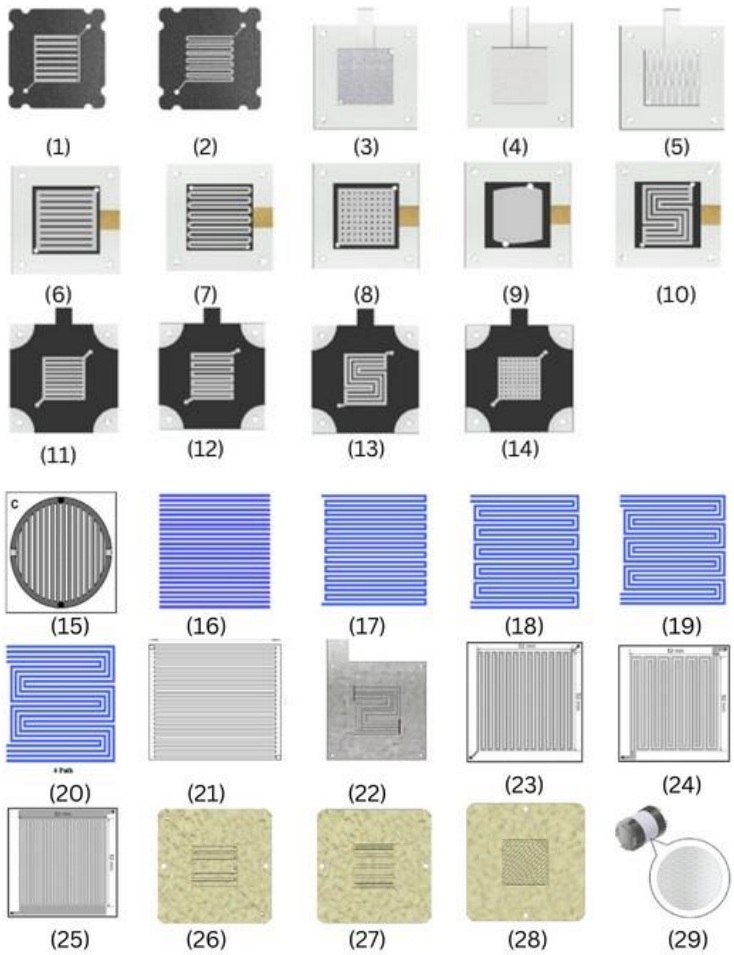


Figure 4 - Flow pattern shapes [1]

3.2 Data Pre-processing

Data preprocessing is a crucial step in the machine learning process, as raw data can often contain missing values, noise and outliers, which can significantly skew the model training and predictive

accuracy [22]. This step can quite literally make or break an ML model. If data is of poor quality, the models could be at the risk of overfitting to the training data. Overfitting occurs when a model essentially memorises the detail and noise in the training data to the point where it performs poorly on new unseen data, hence overfits to the training data. [23].

Initially, a thorough assessment of the data was carried out, which included basic steps such as renaming all the columns in the dataset and setting them to lowercase for consistency. Duplicate entries were removed from the dataset, as the model could learn patterns from these repeated data points, causing overfitting. Removing duplicate entries mitigates this risk. The features in the dataset were then classified into their appropriate types: Category, float64 and int64. Identifying the feature types may seem like a basic step, but this would allow us to explore the use of encoding methods for the categorical features later.

An exploratory data analysis (EDA) was carried out to explore the dataset further, which included generating histograms and boxplots to provide a visual representation of the data distribution, which helped identify skewed columns and outliers in the dataset. A correlation matrix was also plotted, which shows the correlation between different variables. Given the presence of skewed data, feature transformation methods were explored. A logarithmic scaler was applied to reduce the skewness of some distributions and proved effective in this task. The log scaler works by compressing a wide range of skewed data into a smaller one and is particularly effective on right-skewed distributions [24] . As seen below, the effect of the log scaler significantly reduces the skewness of the Power feature.

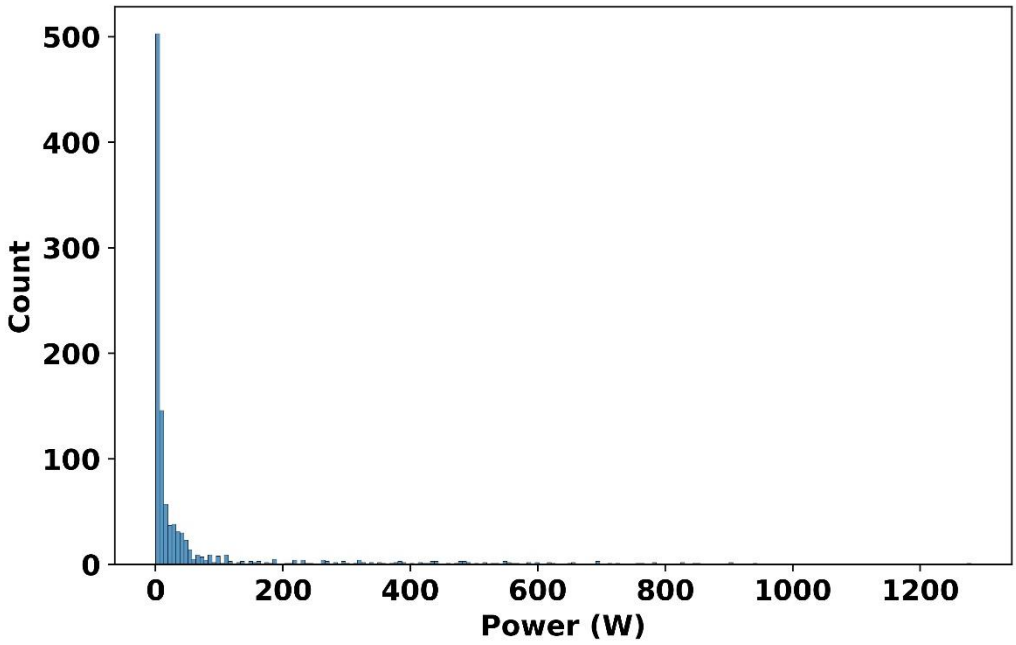


Figure 5 - Histogram of Power (W)

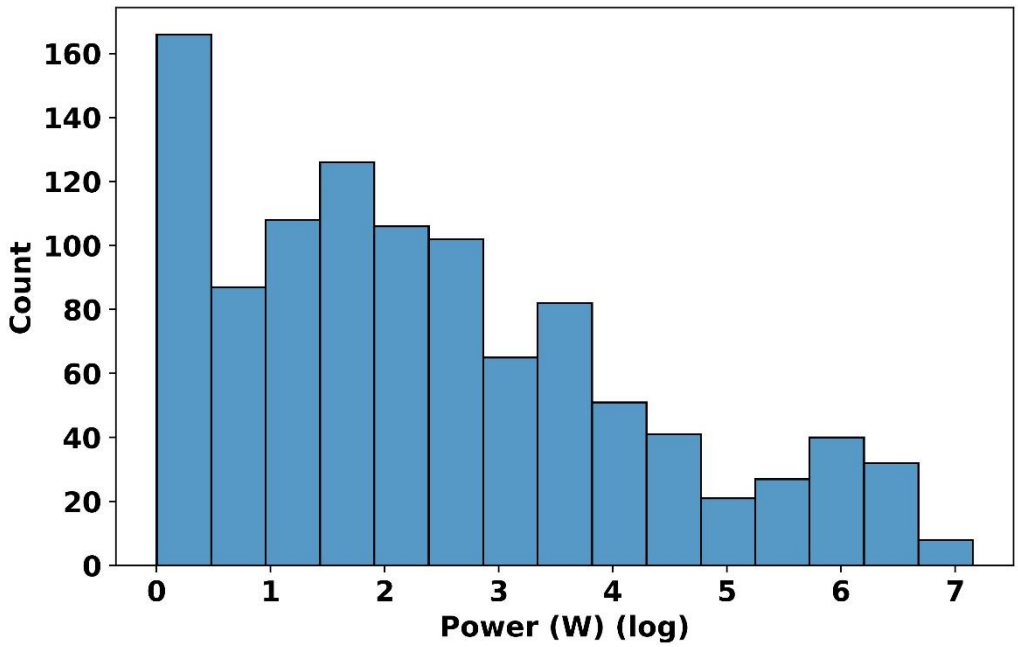


Figure 6 - Histogram of Power (W) with log scaler applied

The numerical variables were then normalised using sci-kit learns standard scaler [25]. This works by using Z-score normalisation, the equation for which is shown below:

$$Z = \frac{X - \mu}{\sigma}$$

Equation 2 - Z-score normalisation [25]

X – The original value of the feature

μ – Mean of the feature across all samples

σ – Standard Deviation of the feature across all samples

Z – The normalised value

Normalisation ensures that features have similar scales and distributions, ensuring that the model does not assign more importance to different features due to their differing scales, improving its accuracy.

After completing all the preprocessing steps, we selected the appropriate input parameters to train our models to predict hydrogen flow rate:

- Electrode area (cm^2)
- Anode/Cathode flow area (cm^2)
- Cell design number
- Cell voltage (V)
- Cell current density (A/cm^2)
- Cell temperature (K)
- Pressure (ATM)
- Anode/Cathode type
- Anode/Cathode GDE
- Membrane type
- Cathode Catalyst
- Anode Catalyst
- Anolyte/Catholyte
- Electrode shape
- Flow type number

Dummy encoding was applied to the categorical variables, which is a method that transforms the categorical variables into a format that the ML algorithms can process.

The data was then split into training and test sets using an 80-20 split. 20% of the data would then be used as a test set to evaluate the performance of the models as unseen data. Unseen

data refers to data that the machine learning models have not been exposed to during their training phase and provides a measure of their ability to generalise from the training data. If the model performs poorly on unseen data, it is likely a sign that the model has overfit to the training data.

3.3 Machine Learning Framework

3.3.1 Software

Python was used for all coding because it offers vast ML and data analysis libraries. Most models used came from the Scikit-Learn library, except for Extreme gradient boosting (XGB), which was imported from a separate library.

3.3.2 Machine Learning Models

All models were used in their regression configuration.

Decision trees (DT) are a supervised ML model that can solve classification and regression tasks. They operate by creating a tree-like structure, starting with the root node. They then split the training data into subsets based on the value of attributes of the data, which is measured by criteria such as entropy information gain and Gini impurity until certain criteria are met, such as the specified maximum depth of the tree. [26] [27]

Random forest (RF) is a supervised ensemble ML model capable of classification and regression tasks and was initially introduced by Breiman in 2001 [28]. It is based on decision trees and operates by creating an ensemble of decision trees using subsets of the dataset chosen randomly to train each tree. The ensemble of trees for classification tasks votes for the most popular output, whereas for regression tasks, the outputs are averaged for the final result. [18] [29]

Extreme gradient boosting (XGB) is a supervised ensemble ML model used for both classification and regression tasks and was initially introduced by Tianqi Chen and Carlos Guestrin in 2016 [30] . It creates an ensemble of weak learners (usually decision trees) and combines their predictions iteratively. Each iteration adds more weak learners to the ensemble, with each new learner focusing on correcting the errors made by the previous weak learners. A gradient-boosting framework technique minimises a predefined loss function during training. Additionally, XGB employs regularisation techniques to enhance model performance and prevent overfitting. [31]

K-nearest neighbours (KNN) is a supervised ML model used for both classification and regression tasks. It operates on the principle of feature similarity, predicting values of new data points based on the category of its 'k' nearest neighbours by calculating the distance between points using metrics such as Euclidian or Manhattan distances. For classification tasks, the predicted output is the most common category amongst the neighbours; for regression tasks, the predicted value is the categories' average values. The choice of the value of k is critical and is usually set to odd numbers to prevent ties in voting. [18] [1]

3.3.3 Hyperparameter Tuning

Hyperparameters refer to the configurations of ML models and are specific to each model. Propper tuning of hyperparameters can significantly improve accuracy and help to prevent overfitting or underfitting. An example of a hyperparameter is the maximum depth of a decision tree, which represents the number of layers, or levels, of nodes from the tree's root to the deepest leaf node [32]. This project used a random search approach with k-fold-cross validation to tune the hyperparameters for all ML models. This approach defines a search space as a set of possible values for each hyperparameter, and the random search algorithm then picks a random combination of hyperparameters. The models are then trained with this configuration of hyperparameters, and their performance is evaluated via cross-validation. This process is repeated a set number of times, and the best-performing hyperparameter combination is returned. [33]

3.4 Model Evaluation

Performance metrics are used in ML to evaluate how a model performs on a particular dataset, helping to determine which model is the most suitable for a specific application.

N – Number of data points in the sample

$\check{y}_i$ – Predicted value from the machine learning model

y_i – Actual value [17]

$\bar{y}$ – Mean of the target variable [34]

Root mean square error:

$$RMSE = \sqrt{(\frac{1}{N}\sum_{i=1}^{N}(y_i - \check{y}_i)^2)}$$

Equation 3 - RMSE

The root mean square error (RMSE) is the root squared sum of the difference between the predicted and actual values—the lower the root mean squared error, the better the model fit.

Mean absolute error:

$$MAE = \frac{1}{N}\sum_{i=1}^{N}|y_i - \check{y}_i|$$

Equation 4 - MAE

The mean absolute error (MAE) is the absolute difference between the predicted and actual values. Similarly to the mean squared error, the lower the mean absolute error, the better the model fit.

R2:

$$R^2 = 1 - \frac{\sum(y_i - \check{y}_i)^2}{\sum(y_i - \bar{y})}$$

Equation 5 - R2

The coefficient of determination (R2) is defined as the proportion of the variance in the dependent variable that is predictable from the independent variables [35]. The value of R2 ranges from 0 to 1, with a value closer to 1 indicating a better fit for the model.

These three metrics will be used to assess which ML model is the best for predicting hydrogen flow rate. The model that returns the lowest combination of errors will be the best-fitting ML model for this application.

3.5 Feature Importance

After the model training was completed, graphs were plotted to show the importance of all the models' features. Feature importance refers to techniques that can be used to calculate a score for all of the input features for a given model, where the scores represent the importance of a feature [36]. The graphs rank the features according to their importance in predicting the target variable. The higher the scores, the more influence a particular feature has on the model's predictive power. Figure 2 shows an example feature importance graph.

There are various techniques for calculating feature importance. Tree-based models, such as DTR, RF and XGB (in its DT configuration) can use Gini importance to calculate feature importance. Gini importance essentially quantifies how much each feature contributes to homogenising the nodes in the trees [36]. The advantage of this method for calculating feature importance is that it is less computationally intense than alternatives such as permutation importance, which models such as KNN require the use of. Permutation feature importance measures the importance of features based on the increase or decrease in errors when the values of the features are randomly shuffled (permuted) [36]. This method is more computationally intense, involving re-evaluating the model multiple times.

3.6 Optimisation of Numerical Parameters

Bayesian optimisation was used to identify the optimal conditions for maximal hydrogen flow rate. This uses the Gaussian process to approximate a function to, in this case, predict hydrogen flow rate, using what is known as a surrogate model [37]. The advantage of this approximation is that it reduces the computational intensity of the optimisation process.

Cell design number	1 – 10
Cell temperature (K)	293.150 – 403.150
Cell voltage (V)	1.38 – 19
Cell current density (A/cm^2)	0.000241 – 2
Electrode area (cm^2)	9 – 100
Anode/Cathode flow area (cm^2)	4 – 50
Pressure (ATM)	1 - 3

Table 4 - Parameter space

Table 4 shows the parameter space set-up based on the bounds of the data used for training the ML models. This informs the surrogate model of the possible values each parameter can take.

The goal was to find the best combination of parameters that maximised the hydrogen flow rate from the best-performing ML model. The optimisation procedure was set to run 50 times, and the result of the Bayesian optimisation provided the optimal set of parameters.

Partial dependence plots can also be plotted to verify the results further and explore the relationships the numerical features have on hydrogen flow rate. These plots are used to visualise the dependence that the target variable (the hydrogen flow rate) has on a single or set of input features (such as pressure) [38].

3.7 Integration with Renewable Energy Source to Forecast Hydrogen Production

Provided we have data for a renewable energy source, we could use this data as an input into the optimised electrolyser model to forecast the amount of green hydrogen we could produce.

We used the publicly available Supervisory Control and Data Acquisition (SCADA) wind turbine dataset to do this [39]. This dataset was taken from a wind turbine at the Yalova wind farm in Turkey. The SCADA system was used to collect metrological data such as wind speed, wind direction and produced power from the period of 1st January to 31st December 2018, representing a year's worth of wind turbine data. The measurements were taken as time series data [40].

Date/Time	LV ActivePower (kW)	Wind Speed (m/s)	Theoretical_Power_Curve (KWh)	Wind Direction (°)
01 01 2018 00:00	380.047791	5.311336	416.328908	259.994904
01 01 2018 00:10	453.769196	5.672167	519.917511	268.641113
01 01 2018 00:20	306.376587	5.216037	390.900016	272.564789
01 01 2018 00:30	419.645905	5.659674	516.127569	271.258087
01 01 2018 00:40	380.650696	5.577941	491.702972	265.674286
…..	…..	…..	…..	…..
31 12 2018 23:50	2820.466064	9.979332	2779.184096	82.274620

Table 5 - Raw SCADA data [39]

Exploring the data revealed that the information was logged every 10 minutes, and 50,530 measurements were taken. For this modelling, the Active power feature was of primary interest, as it represents the actual power produced by the wind turbine, and this feature can be explored further.

	LV ActivePower (kW)
Min	-2.471405
Max	3618.732910

Table 6 – Min and max of LV ActivePower feature

Table 4 shows the minimum and maximum of the Active Power feature. It can be seen that the maximum power produced by the wind turbine was 3.6 MW. Negative values were removed from the dataset, as they could potentially cause our model to inaccurately forecast hydrogen production, reducing the reliability of our prediction. The optimised electrolyser configuraton represents a 19KW electrolyser. So, to satisfy the maximum power values of the wind turbine, a system of 191 19KW 10-cell electrolysers will be required and connected in series. A critical simplifying assumption for this model is that each electrolyser must operate at its total power capacity before any excess power is redirected to the next electrolyser unit in the series. It was also assumed that the electrolysers would be running at a constant power per the SCADA dataset for 10 minutes before switching to a different power output. As the SCADA dataset only provides the power feature, and our model requires an input of voltage and current density, we split the power to satisfy the ratio of 19V per 2 A/cm^2 per cell. The proposed model simulates an off-grid, directly coupled electrolyser renewable energy configuration without any intermediate energy storage device to ease the power fluctuations. It Is also important to note that the model does not account for any additional energy consumption, such as that required for the compression of hydrogen after production.







PEM Electrolysers





Predicted H_2 (Kg)

Figure 7 - Proposed wind energy/hydrogen model. Picture of the Yalova wind farm turbine and the electrolyser graphics were from Shutterstock [41].

4. Results and Discussion

4.1 Correlation Matrix

A correlation matrix is a statistical tool which shows the correlation coefficients between multiple variables, with each row and column representing a different variable. Correlation coefficients range from -1 to +1, where +1 indicates a perfect positive correlation, -1 indicates a perfect negative correlation, and 0 indicates no correlation between the variables. We aim to explore the correlations between different variables related to PEM electrolysis, particularly the highest and lowest correlations, and analyse and explain the relationships observed.

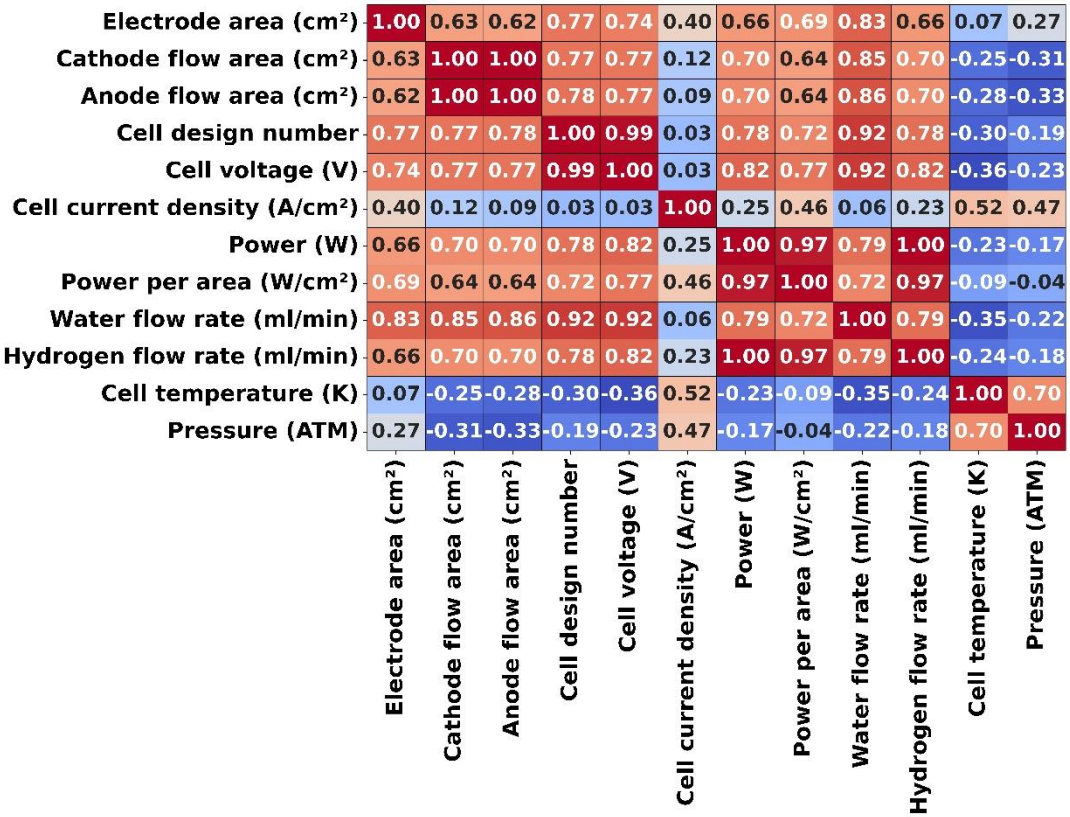


Figure 8 - Correlation matrix for PEM variables

The highest correlation coefficient observed between 2 variables was a value of +1 between the hydrogen flow rate and power feature, indicating a strong positive relationship between these 2 variables. Power is the rate at which an electrical circuit transfers electrical energy, and Faraday's law of electrolysis suggests that the amount of chemical change at an electrode boundary is

proportional to the quantity of electricity used, hence the strong positive correlation between the Hydrogen flow rate and Power across the electrolyser [42].

The lowest correlation coefficient observed between 2 different variables was a value of -0.36 between the Cell voltage and the Cell temperature, indicating a moderately inverse relationship. Increased temperatures can enhance the ionic conductivity of the electrolyte and reduce the overpotential required for electrolysis, resulting in a lower cell voltage at higher temperatures for the same current density [43]. Exploring this relationship further, the cell temperature also has a -0.35 correlation with the water flow rate, suggesting that the cell temperature decreases as the water flow rate increases. The cell voltage has a 0.92 correlation with water flow rate and a 0.82 correlation with hydrogen flow rate. As cell voltage increases, the electrolysis reaction has more of a driving force for hydrogen production, which means more water must be supplied to the cell to match the increased hydrogen production hence the high correlation between cell voltage, hydrogen flow rate and water flow rate. Typically, higher voltages result in higher temperatures in electrolysis, but the increased water flow rate at higher temperatures results in better cell cooling. This is particularly important in PEM electrolysis as higher temperatures can result in the degradation of PEM cell components. For example, elevated temperatures can cause the membrane to dry out, leading to loss of ionic conductivity and thinning of the membrane [44].

4.2 Machine Learning Models Comparison

4 ML models were compared to see which model performed the best when predicting hydrogen production using PEM technologies. We can then explore the relationships between different parameters and hydrogen flow rate upon finding the best-performing model.

Prediction accuracy plots with a regression line were plotted, comparing the predicted values against the actual values. A regression line shows the general trend of a relationship between 2 variables. The closeness of data points to the regression line indicates the strength of the model fit to the data, with a tighter cluster of points around the regression line suggesting a better relationship and fit.

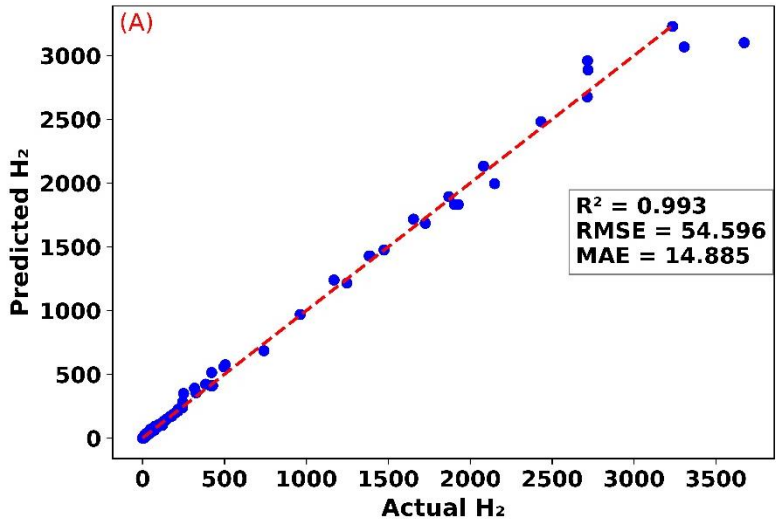


Figure 9a - Prediction accuracy scatter plot with regression line for XGB

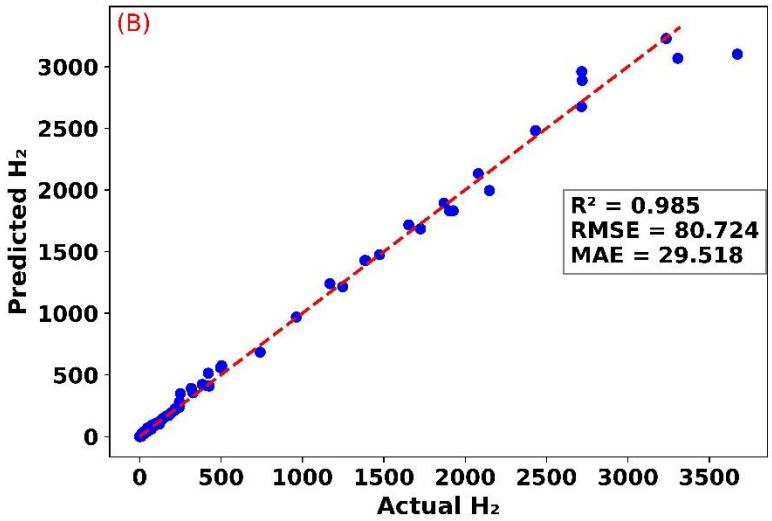


Figure 9b - Prediction accuracy plot with regression line for DTR

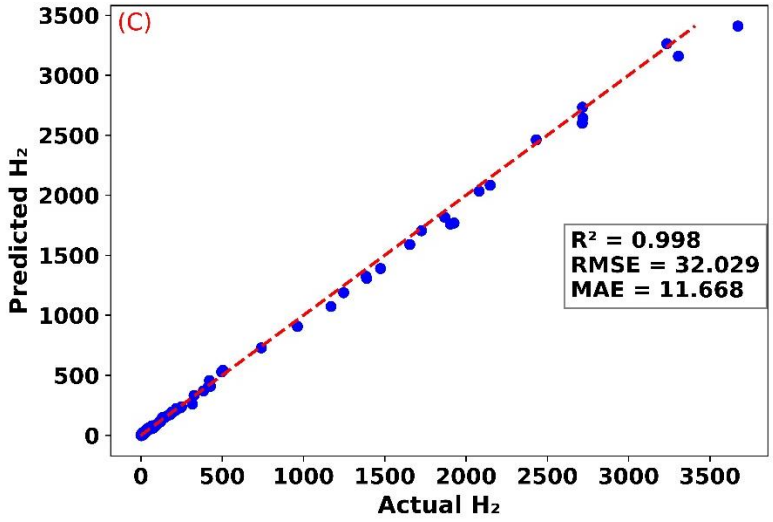


Figure 9c - Prediction accuracy plot with regression line for RF

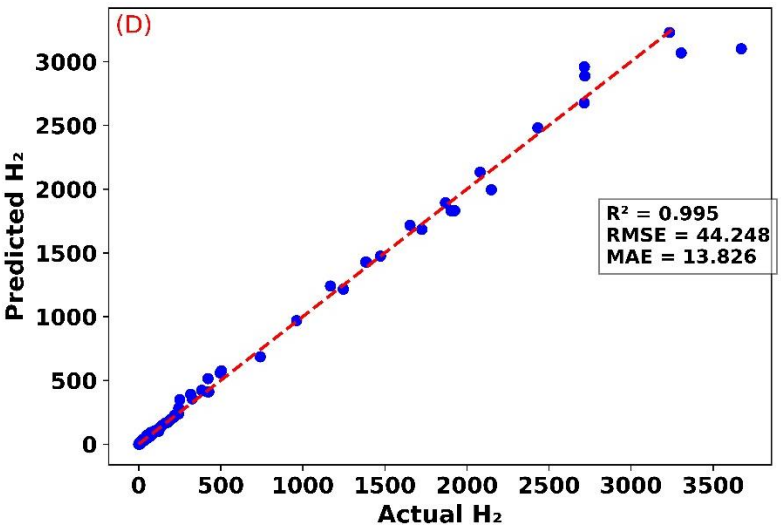


Figure 9d - Prediction accuracy plot with regression line for KNN

3 metrics were used to evaluate model performance: R2, RMSE and MAE. In Figure 9, the distribution of data points and trends in all graphs appear very similar, suggesting that all models effectively predict the hydrogen flow rate for this dataset. The RMSE and MAE values are relatively low for all models, suggesting that the predictions are close to the actual values for all models. However, of all models, RF exhibited the highest predictive accuracy and the lowest errors, with values of 0.998, 32.029, and 11.668 for R2, RMSE, and MAE. This greater accuracy can be attributed to RF being particularly adept at modelling intricate and non-linear relationships among variables, which is highly valuable in the context of PEM electrolysis, where

it can effectively model the sophisticated and non-linear associations that exist between variables like current density, temperature, pressure, and hydrogen flow rate [18]. KNN was the second-best performing model with R2, RMSE, and MAE values of 0.995, 44.248, and 13.826. In this case, KNN's reliance on the distance between data points indicating their similarity proves effective. The third best-performing model was XGB, with R2, RMSE, and MAE values of 0.993, 54.596, and 14.885. XGB is quite sensitive to hyperparameters, and different combinations can significantly affect XGB's predictive power and ability to learn and generalise from the data [45]. The hyperparameter tuning method used was Randomized search. Its process of randomly selecting a subset of parameters could miss the optimal combination of hyperparameters which could be why XGB is underperforming. The least effective model was DTR, with R2, RMSE, and MAE values of 0.985, 80.724, and 29.518.

4.3 Feature Importance Comparison

Feature importance graphs are visual tools that indicate the relative importance input features have on the model's predictive power. We can use feature importance graphs to explore what features the models believe to be most influential on hydrogen flow rate. Our categorical variables have been dummy-coded, and this can also be used to find the most optimal design for the PEM electrolyser. Figure 10 shows the feature importance graphs for this study's 4 different ML models.

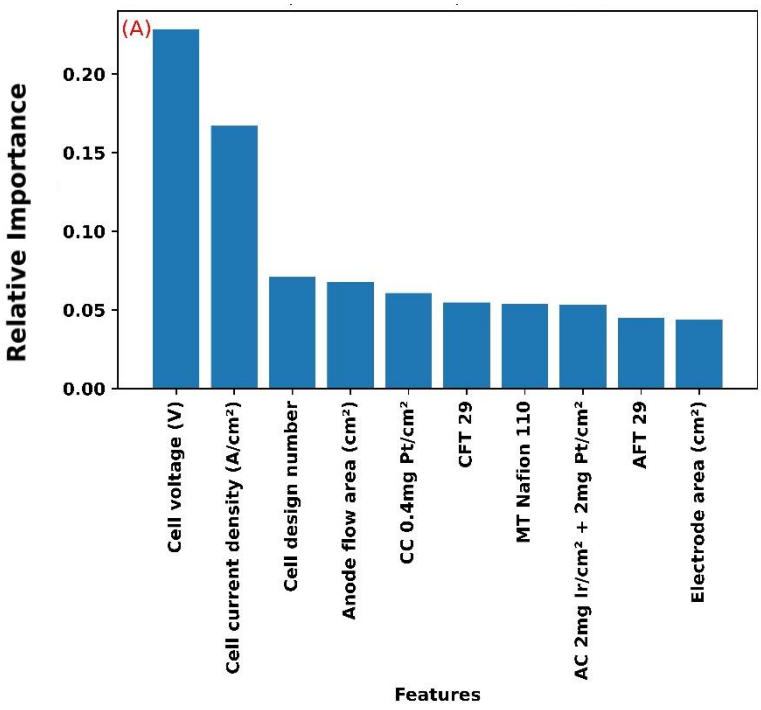


Figure 10a – Top 10 feature importance graph for RF

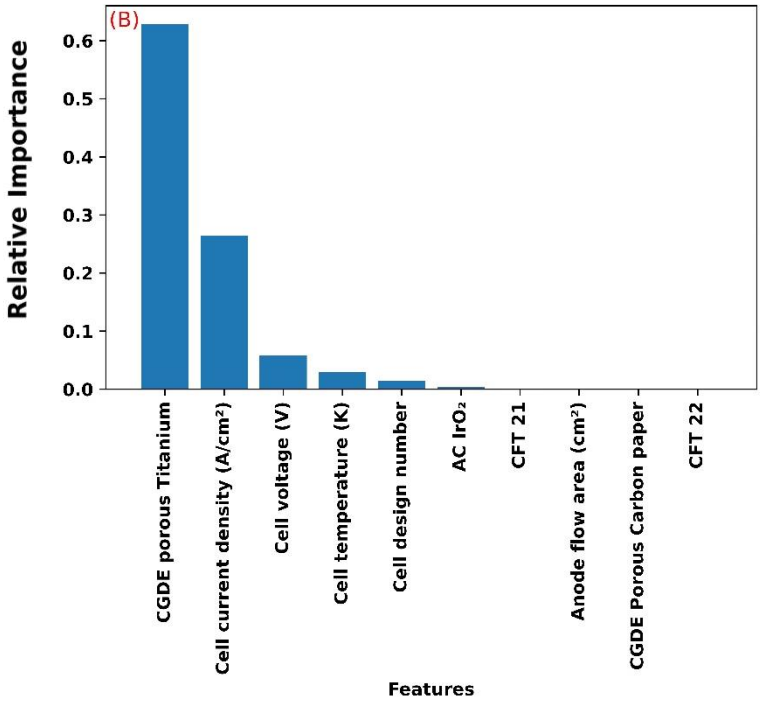


Figure 10b – Top 10 feature importance graph for DTR

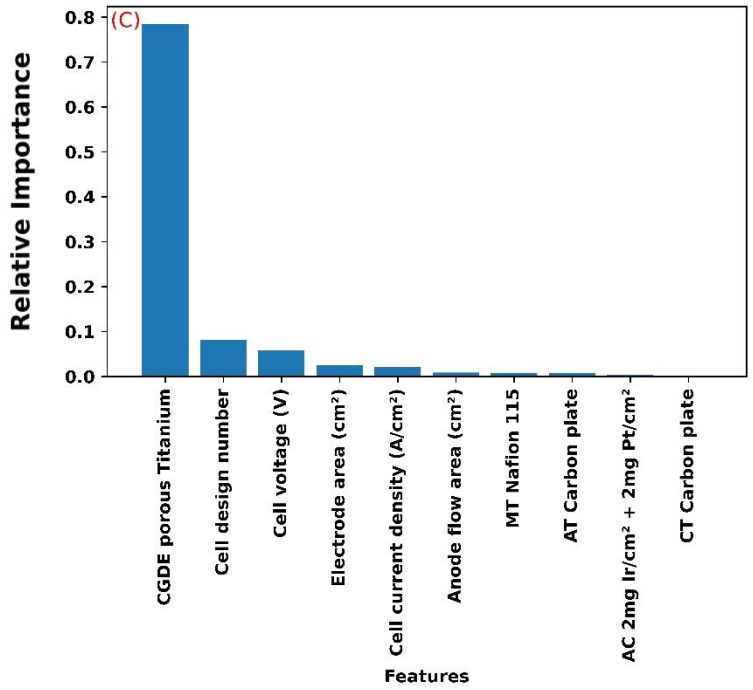


Figure 10c – Top 10 feature importance graph for XGB

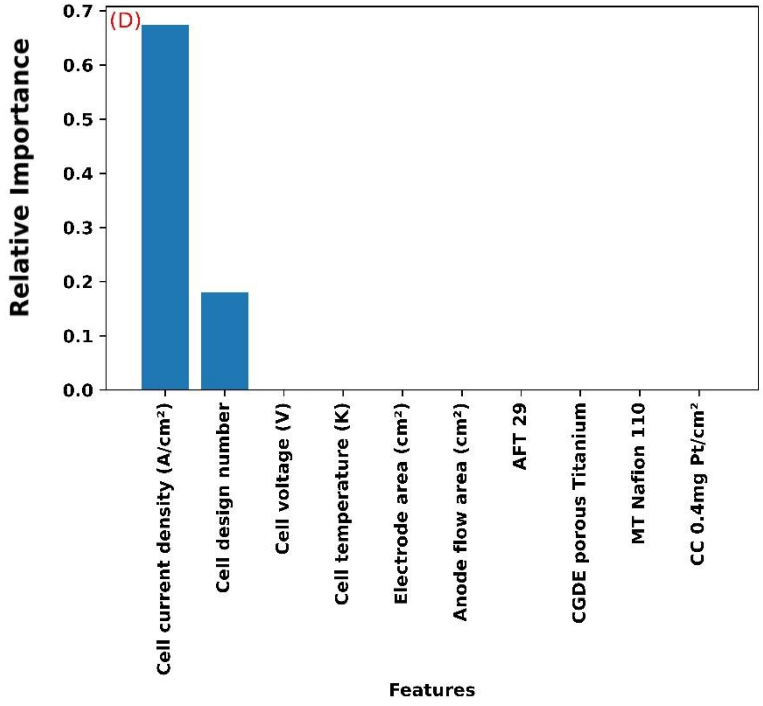


Figure 10d – Top 10 feature importance graph for KNN

Figure 10 shows little consistency between the feature importance for the different models. Both DTR and XGB rank the Cathode Gas diffusion electrode (GDE) porous Titanium as the top

feature for predicting hydrogen flow rate, suggesting these models believe this feature to be the most influential for predicting hydrogen flow rate. In contrast, RF ranks the cell Voltage as the most influential, and KNN ranks Cell current density as the most influential. Despite this variation in feature importance, the low errors all the models exhibit suggest the models are still capturing all essential patterns in the data. The differences could be attributed to the different ways the models learn data complexity, and each model may capture different aspects of the underlying data structure, hence the differing feature importances. There are, however, a few features which maintain higher rankings consistency amongst the models; these are the cell voltage, the cell current density, and the cell design number (number of cells in the electrolyser). These features consistent appearance at the higher ranks suggests they contribute significantly to the hydrogen flow rate in PEM electrolysis.

Random forest was considered to be the most accurate and demonstrated more distributed feature importance, indicating more features significantly influenced the hydrogen flow rate compared to other models, so this can be explored further:

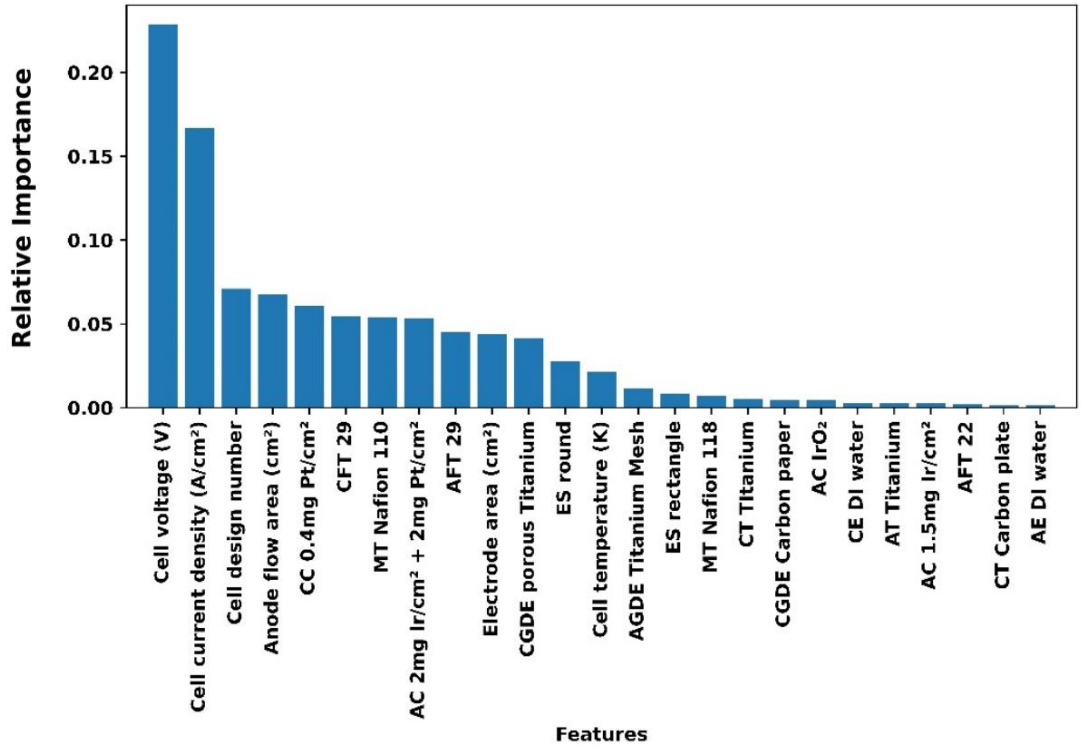


Figure 11 - RF top 25 feature importance graph

4.3.1 Process Parameter Importance

In Figure 11, Random Forest ranked the Cell voltage as the most influential process parameter. The cell voltage provides the driving force for the electrochemical reaction to split water. The minimum required voltage is known as the Thermo neutral voltage, and the cell voltage must exceed this for the splitting of water molecules [9]. Increasing voltage increases the charge density on electrode surfaces, which facilitates a stronger electron discharge, speeding up the water decomposition reaction as there is more energy to break the molecular bonds in water. [46]. This is concurrent with Faraday's law of electrolysis, which suggests that the amount of chemical change at an electrode boundary (in this case, hydrogen), is proportional to the quantity of electricity used [42]. By increasing the voltage, the current also increases, which results in more hydrogen being produced. The cell current density refers to the amount of electric current flowing over the area of the electrode, and greater current densities mean there are more electrons available for the water splitting reaction, specifically more availability of electrons at the cathode for reducing water molecules into hydrogen. The cell voltage and current density directly influence the reaction kinetics of water electrolysis, hence their higher rankings in the feature importance chart.

The third highest process parameter was the cell design number, representing the number of cells in the electrolyser configuration. The number of cells in an electrolyser directly affects the hydrogen flow rate. Increasing the number of cells allows for more total electrode surface area, increasing the number of sites where electrochemical reactions can occur and thus greatly influencing the hydrogen production rate [47]. Each cell in an electrolyser typically has a set voltage based on its chemistry and design. Stacking multiple cells in an electrolyser adds up their voltages, allowing for a higher total voltage and, thus, a greater hydrogen flow rate.

The fourth-ranked process parameter was the anode/cathode flow area. This refers to the cross-sectional area of the machined channels which distribute water into the PEM cell and carry the generated gases to the outlet of the PEM system [48]. Generally, the flow area can directly affect the hydrogen flow rate, as smaller areas could constrict hydrogen flow out of the cell, so optimal flow areas must be used for water and generated gases for efficient flow into and out of the cell.

The fifth-ranked process parameter was the Electrode Area. The electrode area refers to the area of the electrode which is directly involved in the water-splitting reaction for hydrogen

production. The more electrode surface area available, the more sites there are for the electrochemical reaction of water splitting, resulting in a higher current density [18]. Also, in PEM electrolysers, electrodes are usually coated with catalysts, and a larger electrode area allows for more catalysts to be utilised in the reaction, showing how the electrode area also significantly influences the reaction kinetics of PEM electrolysers.

The sixth-ranked process parameter was the Cell temperature. The cell temperature significantly influences the efficiency and performance of a PEM electrolyser cell. Higher temperatures tend to improve reaction kinetics and can reduce electrical resistance in the cell, leading to higher hydrogen production rates [49]. At electrodes, this means quicker ion transport and more efficient water-splitting [18].

The cell pressure had the lowest feature importance out of all process parameters, and the RF algorithm and all other algorithms did not consider it to influence the hydrogen flow rate much.

The rankings' validity cannot be entirely commented upon as this is such a novel approach. However, the rankings appear logical and align with electrochemical principles. As previously stated, feature importance refers to the relative importance input features have on the model's predictive power, which may differ from an engineering perspective.

4.3.2 Optimal Design

The feature importance graphs also provide which categorical features are the most influential in the prediction of hydrogen production. In this case, it helps with the electrolyser's best material parameters and optimal design. The optimal material parameters and design of the electrolyser taken from Figure 11 are shown in Table 7:

Anode Type	Titanium
Cathode Type	Titanium
Anode GDE (Gas diffusion electrode)	Titanium mesh
Cathode GDE (Gas diffusion electrode)	Porous Titanium
Membrane Type	Nafion 110
Cathode catalyst	0.4mg Pt/cm^2
Anode catalyst	2 mg of Pt/cm^2 and 2 mg of IrO_2/cm^2

Catholyte	Deionised water
Anolyte	Deionised water
Electrode shape	Round
Anode flow type	 [1]
Cathode flow type	 [1]

Table 7 - Optimal design parameters

In PEM electrolysis, using PFSA membranes, high voltages, and the oxygen evolution reaction at the anode creates a harsh oxidative and acidic environment, especially prevalent at higher hydrogen flow rates. Materials which are capable of withstanding these conditions are required [10]. At higher hydrogen flow rates, titanium-based materials are generally favoured, as Titanium offers exceptional corrosion resistance, which helps to provide long-term stability to the cell components in PEM electrolysis. Hence, the selection of titanium-based materials as optimal is plausible.

The most optimal membrane was selected as the Nafion 110 membrane. Nafion membranes tend to be used in PEM electrolysers due to advantages such as being able to operate at higher current densities (up to 2 A/cm^2), high durability, high proton conductivity and good mechanical stability. [9]. These advantages are essential, especially in larger-scale applications, where Nafion membrane's durability is particularly advantageous as these membranes can resist degradation from the continuous operation at higher current densities and the reactive element within PEM cells.

Iridium oxide and Platinum are generally used as catalysts in PEM electrolysis, mainly due to their high catalytic activity and stability under the oxygen evolution reaction and hydrogen evolution reaction conditions [50]. This high catalytic activity allows for the reaction to happen at lower overpotentials, allowing for enhanced efficiency and increased hydrogen flow rate, and

both catalysts exhibit outstanding stability in the harsh acidic environment generated by the PFSA membrane [9].

4.4 Optimal Process Parameters

Cell design number	Cell temperature (K)	Cell voltage (V)	Cell current density (A/cm^2)	Electrode area (cm^2)	Anode/cathode flow area (cm^2)	Pressure (ATM)
10	337.03	19.00	2.00	80.86	50.00	1.00

Table 8 - Bayesian optimisation results for maximising hydrogen flow rate

Table 8 shows the numerical parameters that the optimisation algorithm decided as optimal for the maximum hydrogen flow rate. The cell design number, voltage, current density, and the anode/cathode flow area values returned were all the maximum bounded values available to the algorithm. The cell temperature, electrode area, and pressure returned values that were lower than the maximum. To explore the relationships these parameters have on the hydrogen flow rate further, we can use partial dependence plots.

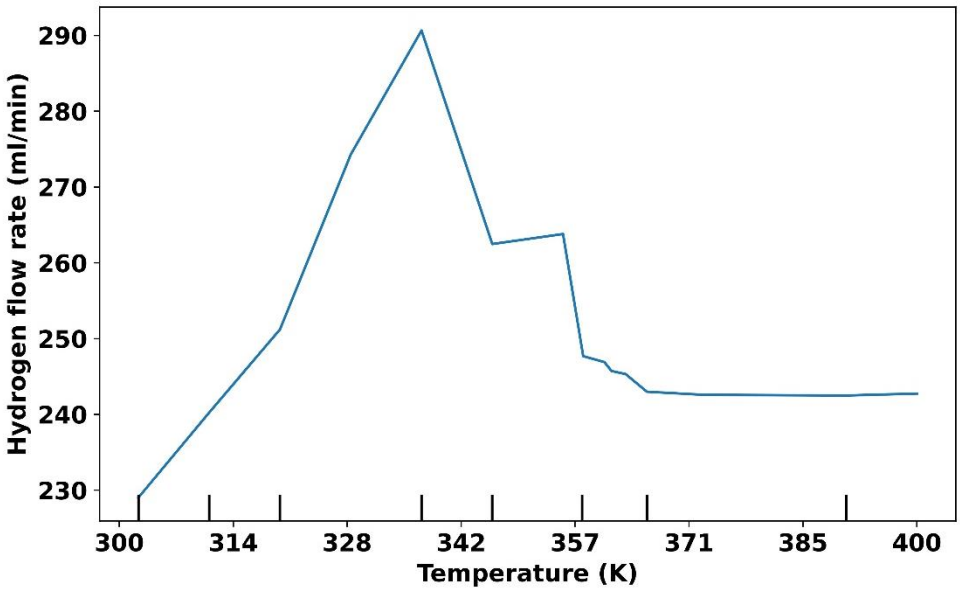


Figure 12 - Partial dependence plot of hydrogen flow rate on temperature

Figure 12 shows a partial dependence plot of hydrogen flow rate on temperature and reveals an optimal temperature range. The optimisation algorithm returned a value of 337 K as the optimal

for hydrogen production, and referring to Figure 12, this is concurrent with the maximum point of the graph. As temperature increases past this optimum value, we observe a decrease in the hydrogen flow rate and cell temperatures above 356 K result in lower hydrogen production. Higher temperatures can cause dehydration of the PFSA membranes, resulting in a loss of ionic conductivity and significant thinning of the membranes, leading to degradation of the PEM cell [44, 51]. This further impacts the performance of electrolysis and, hence, the presence of an optimal temperature range.

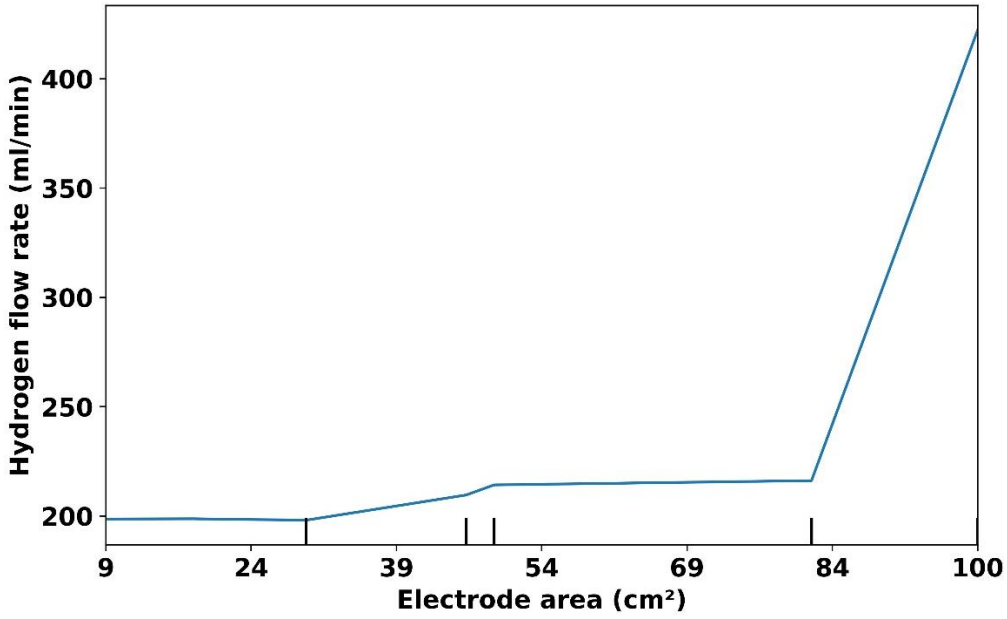


Figure 13 - Partial dependence plot of electrode area on hydrogen flow rate

Figure 13 shows a partial dependence plot of electrode area on hydrogen flow rate. Increases in electrode area up to 82 cm^2 do not seem to increase the hydrogen flow rate much. Values above 82 cm^2 appear to cause a particularly sharp rise in the hydrogen flow rate up to an electrode area of 100 cm^2. The optimisation algorithm returned a value of 80.86cm^2 as optimum for the electrode area, which is not concurrent with Figure 13. This is likely due to the interdependencies the electrode area has with the other variables. Higher electrode areas could negatively interact with different variables, resulting in diminished hydrogen production, hence the optimal of 80.86cm^2.

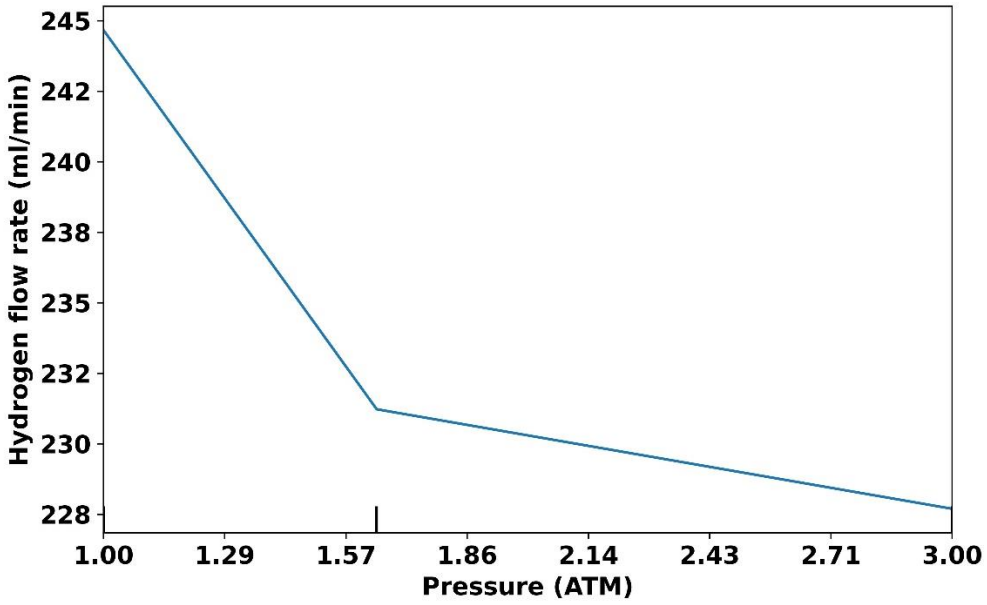


Figure 14 - Partial dependence plot of pressure on hydrogen flow rate

Referring to Figure 14, there appears to be an inverse relationship between pressure and hydrogen flow rate, suggesting that higher pressure conditions are associated with a reduction in the hydrogen flow rate within the studied range. The optimisation algorithm chose a pressure of 1 ATM as the optimal, which is concurrent with the partial dependence plot. The reduction in hydrogen flow rate appears to be more pronounced between 1 ATM and 1.6 ATM; beyond this, the hydrogen flow rate continues to decrease but at a much less steep rate. Higher cell pressures generally do increase electrolysis kinetics, but excessive pressures can result in mechanical stresses, gas leakages, and in some cases, membrane damage, so pressure must be optimised for optimal hydrogen production [18].

4.5 Optimal Design and Comparison with Previous Study

A)		B)	
Anode Type	Titanium	Anode Type	Titanium
Cathode Type	Titanium	Cathode Type	Titanium
Anode GDE	Titanium mesh	Anode GDE	Titanium mesh
Cathode GDE	Porous Titanium	Cathode GDE	Porous Titanium
Membrane Type	Nafion 110	Membrane Type	Nafion 110
Cathode catalyst	0.4mg Pt/cm^2	Cathode catalyst	0.4mg Pt/cm^2

Anode catalyst	2 mg of Pt/cm^2 and 2 mg of IrO_2/cm^2	Anode catalyst	2 mg of Pt/cm^2 and 2 mg of IrO_2/cm^2
Catholyte	Deionized water	Catholyte	Deionized water
Anolyte	Deionized water	Anolyte	Deionized water
Electrode shape	Round	Electrode shape	NA
Anode flow type	 [1]	Anode flow type	 [1]
Cathode flow type	 [1]	Cathode flow type	 [1]
Cell design number	10	Cell design number	10
Cell temperature (K)	337.03	Cell temperature (K)	333.15
Cell voltage (V)	19	Cell voltage (V)	17.11
Cell current density (A/cm^2)	2	Cell current density (A/cm^2)	0.87
Electrode area (cm^2)	80.86	Electrode area (cm^2)	100
Anode/cathode flow area (cm^2)	50	Anode/cathode flow area (cm^2)	100
Pressure (ATM)	1	Pressure (ATM)	1
Hydrogen flow rate (ml/min)	4200	Hydrogen flow rate (ml/min)	3000

Table 9 - Comparison between A) the results of this study and B) the results of a previous study [1]

To validate our results, we can compare the results we have with the previous study, which utilised this dataset. Its model was able to predict various electrolyser design parameters based on a set of 5 input parameters: hydrogen production rate, electrode area, anode and cathode flow areas, and cell design type [1]. Their model could predict with a 50-3000 ml/min hydrogen production range. The approach utilised in this study was completely different. We predicted the hydrogen flow rate using all other features as input parameters. We applied optimisation and feature importance techniques to find the most optimal electrolyser operating conditions and material combinations for maximising hydrogen flow rate. At the previous studies' highest hydrogen flow rate of 3000 ml/min, the material types and flow types remained consistent with

our study, validating our results. Our study had differing voltage, temperature, and current density, which is viable as the returned hydrogen flow rate was also greater than the previous study's. As both studies used inherently different approaches, one approach cannot be ranked above the other. Instead, the results can be used for validation.

4.6 Hydrogen Production Forecasting Based on Dynamic Wind Energy Inputs

In this section, we explore the integration of wind energy into our optimised electrolyser system. To investigate the relationship between simulated hydrogen production and dynamic energy inputs, we can explore the power and hydrogen produced on a random day, the 4th of February, 2018.

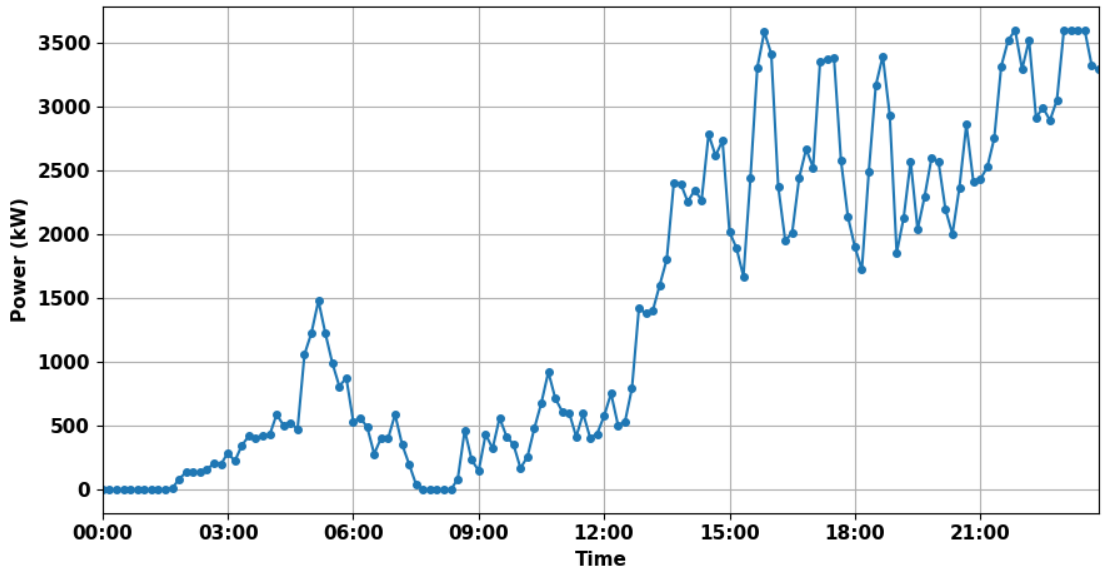


Figure 15 - Dynamic wind energy profile for 4th February 2018.

Figure 20 shows the dynamic wind energy profile for the 4th of February 2018. There is significant fluctuation of power produced throughout the day. The peaks and troughs observed likely correspond with the wind intensity at different times of day, which leads to high and low power production periods. This plot highlights the significant variability of wind power.

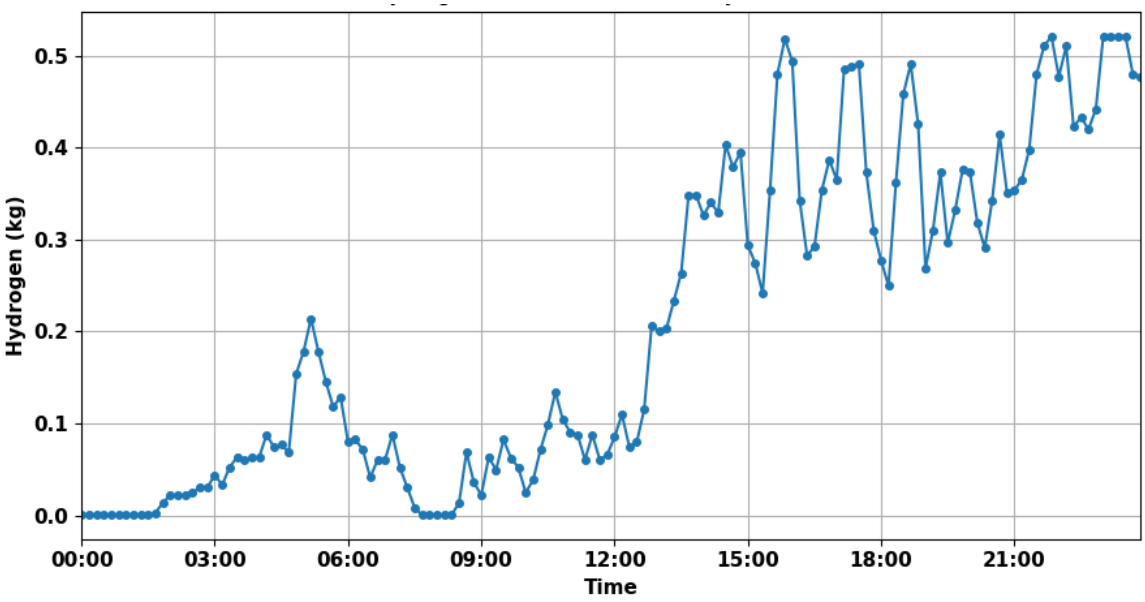


Figure 16 - Hydrogen production throughout the day on February 4th 2018

Figure 21 shows the hydrogen produced throughout the day on the 4th of February, 2018; as expected, there is significant fluctuation throughout the day. The hydrogen production pattern closely follows the power fluctuation pattern presented in Figure 20. This highlights the dependency of green hydrogen production on the intermittent nature of renewable energy, especially when electrolysers are directly coupled to a renewable power source.

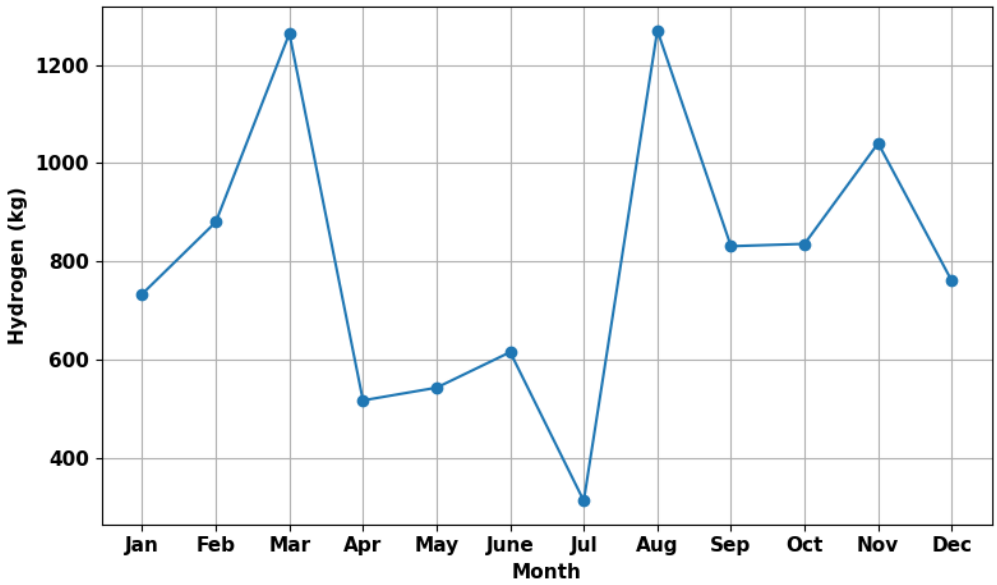


Figure 17 - Graph of hydrogen produced each month.

Figure 22 shows the total hydrogen produced each month over the year. There is a significant fluctuation in hydrogen production throughout the year, with production peaking in February and August. This could be due to stronger wind conditions in these months, and the lowest hydrogen production occurs in July. The daily hydrogen production grouped by month, and the dynamic power profile grouped by month is shown in Appendix A.

The simulation successfully forecasted 10 tonnes of hydrogen produced throughout the year using the optimised electrolyser model based on the power output from a wind turbine at the Yalova wind farm in Turkey. Notably, the forecasted production of 10 tonnes of hydrogen represents an output under ideal conditions. It assumes a seamless energy transfer from wind power to the electrolysers without accounting for potential losses that could occur in a real-world setting.

The value of 10 tonnes over the year does seem to be a low number for coupling with a 3.6 MW wind turbine. Comparing this figure to that of Torbjørn Egeland-Eriksen et al., who forecasted 17 tonnes of hydrogen over a specific 31-day period with a 1.852 MW electrolyser from a 2.3 MW offshore wind turbine, it is clear that our forecasted hydrogen production is deficient over the year [14]. The nature of wind is very variable, and less power is likely to be produced from an onshore wind turbine as opposed to an offshore one, as the wind velocity at offshore locations is generally higher and more consistent, but this does not entirely explain the much lower hydrogen yield over the year [52]. It becomes clear that our system of optimised 19KW electrolysers is not suitable for hydrogen production at such a large scale or integration with renewable energy.

5. Conclusion

In conclusion, the main aims and objectives of this study were satisfied. We constructed and compared ML models to predict the hydrogen flow rate based on 15 input parameters. Our model evaluation found that RF was the best-performing model for predicting hydrogen flow rate for PEM electrolyser technologies. We also performed a feature importance analysis to find which process parameters most influenced the hydrogen flow rate. We found that the cell voltage, cell current density, and cell design number were consistently highly ranked amongst the feature importance graphs. Therefore, according to our models, they are the most influential factors for hydrogen flow rate. We then utilised feature importance analysis and optimisation techniques to find the optimal design and operational conditions for the electrolyser to maximise the hydrogen flow rate, and we succeeded at this. We then integrated our optimised electrolyser model with a renewable energy source and forecasted an overall hydrogen production of 10 tonnes throughout the year. However, we found that the optimised electrolyser design was unsuitable for large-scale hydrogen production through renewable energy due to the low hydrogen yield. In summary, our study shows the potential ML techniques have in addressing significant issues related to green hydrogen production and the critical role this technology will have in advancing the hydrogen economy.

6. Limitations and Future Directions

This study does have some limitations. A significant limitation was that the dataset was small and consisted of smaller-scale data. We would benefit from repeating the study, perhaps with mega-watt scale data, and using optimisation and ML techniques at this scale to optimise electrolyser design. Additionally, comparing more advanced ML models, such as ANN, would be beneficial in finding out which model is best suited for predicting hydrogen production through PEM technologies. The optimal design should be practically created to assess and validate the results.

The hydrogen forecasting simulation oversimplifies the real-life scenario and ignores potential losses and inefficiencies. For example, we did not consider the energy required to power the compressors for hydrogen storage. Using a machine learning model with optimised parameters to forecast hydrogen production is an effective way of predicting, and this method could be explored further. The model could be adapted to use renewable energy data from different geographical locations to forecast the amount of hydrogen that could be produced at various locations. Helping to assess which sites are most optimal for green hydrogen production. Perhaps even going as far as using ML to forecast environmental parameters (as explored in 3.5.2), and in place of mathematical modelling, using an ML model to simulate an electrolyser for forecasting hydrogen production and coupling both models together. The experiment could also be repeated by coupling an energy storage device to manage the intermittent renewable energy source.

7. Bibliography

[1] R. Yang, A. Mohamed, and K. Kim, "Optimal design and flow-field pattern selection of proton exchange membrane electrolyzers using artificial intelligence," *Energy,* vol. 264, p. 126135, 2023, doi: 10.1016/j.energy.2022.126135.

[2] M. M. Kabir, M. M. Akter, Z. Huang, L. Tijing, and H. K. Shon, "Hydrogen production from water industries for a circular economy," *Desalination,* vol. 554, p. 116448, 2023, doi: 10.1016/j.desal.2023.116448.

[3] C. Acar and I. Dincer, "The potential role of hydrogen as a sustainable transportation fuel to combat global warming," *International Journal of Hydrogen Energy,* vol. 45, no. 5, pp. 3396-3406, 2020, doi: 10.1016/j.ijhydene.2018.10.149.

[4] PricewaterhouseCoopers. "Green hydrogen economy - predicted development of tomorrow." https://www.pwc.com/gx/en/industries/energy-utilities-resources/future-energy/green-hydrogen-cost.html (accessed 10/05/24.

[5] A. M. Oliveira, R. R. Beswick, and Y. Yan, "A green hydrogen economy for a renewable energy society," *Current Opinion in Chemical Engineering,* vol. 33, p. 100701, 2021, doi: 10.1016/j.coche.2021.100701.

[6] E. B. Agyekum, C. Nutakor, A. M. Agwa, and S. Kamel, "A Critical Review of Renewable Hydrogen Production Methods: Factors Affecting Their Scale-Up and Its Role in Future Energy Generation," *Membranes,* vol. 12, no. 2, p. 173, 2022, doi: 10.3390/membranes12020173.

[7] S. Sebbahi, N. Nabil, A. Alaoui-Belghiti, S. Laasri, S. Rachidi, and A. Hajjaji, "Assessment of the three most developed water electrolysis technologies: Alkaline Water Electrolysis, Proton Exchange Membrane and Solid-Oxide Electrolysis," *Materials Today: Proceedings,* vol. 66, pp. 140-145, 2022/01/01/ 2022, doi: https://doi.org/10.1016/j.matpr.2022.04.264.

[8] BYJU. "Electrolysis." https://byjus.com/jee/electrolysis/ (accessed 05/04/2024.

[9] S. Shiva Kumar and V. Himabindu, "Hydrogen production by PEM water electrolysis – A review," *Materials Science for Energy Technologies,* vol. 2, no. 3, pp. 442-454, 2019, doi: 10.1016/j.mset.2019.03.002.

[10] IRENA, "Green Hydrogen Cost Reduction: Scaling up Electrolysers to Meet the 1.5°C Climate Goal." [Online]. Available: /-/media/Files/IRENA/Agency/Publication/2020/Dec/IRENA_Green_hydrogen_cost_2020.pdf?rev=95b8c10569874148a44e1d17b301d263

[11] S. Shiva Kumar and H. Lim, "An overview of water electrolysis technologies for green hydrogen production," *Energy Reports,* vol. 8, pp. 13793-13813, 2022, doi: 10.1016/j.egyr.2022.10.127.

[12] A. Mohammadi and M. Mehrpooya, "A comprehensive review on coupling different types of electrolyzer to renewable energy sources," *Energy,* vol. 158, pp. 632-655, 2018, doi: 10.1016/j.energy.2018.06.073.

[13] N. Srettiwat, M. Safari, H. Olcay, and R. Malina, "A techno-economic evaluation of solar-powered green hydrogen production for sustainable energy consumption in Belgium," *International Journal of Hydrogen Energy,* vol. 48, no. 100, pp. 39731-39746, 2023, doi: 10.1016/j.ijhydene.2023.09.159.

[14] T. Egeland-Eriksen, J. F. Jensen, Ø. Ulleberg, and S. Sartori, "Simulating offshore hydrogen production via PEM electrolysis using real power production data from a 2.3 MW floating offshore wind turbine," *International Journal of Hydrogen Energy,* vol. 48, no. 74, pp. 28712-28732, 2023, doi: 10.1016/j.ijhydene.2023.03.471.

[15] S. Gamesa. "Green hydrogen." https://www.siemensgamesa.com/en-int/products-and-services/hybrid-and-storage/green-hydrogen (accessed 26/04/2024.

[16] IBM. "What is machine learning (ML)?" ibm.com. https://www.ibm.com/topics/machine-learning (accessed 10/03/24.
[17] A. Mohamed, H. Ibrahem, R. Yang, and K. Kim, "Optimization of Proton Exchange Membrane Electrolyzer Cell Design Using Machine Learning," *Energies,* vol. 15, no. 18, p. 6657, 2022, doi: 10.3390/en15186657.
[18] M. M. Kabir *et al.*, "Machine learning-based prediction and optimization of green hydrogen production technologies from water industries for a circular economy," *Desalination,* vol. 567, p. 116992, 2023, doi: 10.1016/j.desal.2023.116992.
[19] A. Mohamed, H. Ibrahem, and K. Kim, "Machine learning-based simulation for proton exchange membrane electrolyzer cell," *Energy Reports,* vol. 8, pp. 13425-13437, 2022, doi: 10.1016/j.egyr.2022.09.135.
[20] A. A. Alhussan *et al.*, "Green hydrogen production ensemble forecasting based on hybrid dynamic optimization algorithm," *Frontiers in Energy Research,* vol. 11, 2023, doi: 10.3389/fenrg.2023.1221006.
[21] S. A. Haider, M. Sajid, and S. Iqbal, "Forecasting hydrogen production potential in islamabad from solar energy using water electrolysis," *International Journal of Hydrogen Energy,* vol. 46, no. 2, pp. 1671-1681, 2021, doi: 10.1016/j.ijhydene.2020.10.059.
[22] M. Kang and J. Tian, "Machine Learning: Data Pre-processing," John Wiley and Sons Ltd, 2018, pp. 111-130.
[23] X. Ying, "An Overview of Overfitting and its Solutions," *Journal of Physics: Conference Series,* vol. 1168, p. 022022, 2019, doi: 10.1088/1742-6596/1168/2/022022.
[24] Changyong FENG, *,* Hongyue WANG,1 Naiji LU,1 Tian CHEN,1 Hua HE,1 Ying LU,2 and Xin M. TU1, "Log-transformation and its implications for data analysis," *Shanghai Arch Psychiatry.,* pp. 105–109., 2019, doi: https://doi.org/10.3969%2Fj.issn.1002-0829.2014.02.009.
[25] L. Scikit. "sklearn.preprocessing.StandardScaler — scikit-learn 0.21.2 documentation." https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html (accessed 01/05/2024.
[26] GeeksForGeeks. "Decision Tree - GeeksforGeeks." https://www.geeksforgeeks.org/decision-tree/ (accessed 02/05/24.
[27] T. Thomas, A. P. Vijayaraghavan, and S. Emmanuel, "Applications of Decision Trees," Springer Singapore, 2020, pp. 157-184.
[28] L. Breiman, *Machine Learning,* vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/a:1010933404324.
[29] E. R. Sruthi. "Random Forest | Introduction to Random Forest Algorithm." https://www.analyticsvidhya.com/blog/2021/06/understanding-random-forest/ (accessed 26/04/2024.
[30] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," 2016. [Online]. Available: https://dmlc.cs.washington.edu/data/pdf/XGBoostArxiv.pdf
[31] A. VIdha. "Understanding the Math behind the XGBoost Algorithm." https://www.analyticsvidhya.com/blog/2018/09/an-end-to-end-guide-to-understand-the-math-behind-xgboost/ (accessed 22/04/2024.
[32] scikit-learn. "sklearn.ensemble.RandomForestRegressor — scikit-learn 0.20.3 documentation." https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html (accessed 26/04/2024.
[33] l. scikit. "sklearn.model_selection.RandomizedSearchCV — scikit-learn 0.21.3 documentation." https://scikit-

learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html (accessed 03/05/2024.

[34] D. Jain. "ML | R-squared in Regression Analysis - GeeksforGeeks." https://www.geeksforgeeks.org/ml-r-squared-in-regression-analysis/ (accessed 10/03/2024.

[35] D. Chicco, M. J. Warrens, and G. Jurman, "The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation," *PeerJ Computer Science,* vol. 7, p. e623, 2021, doi: 10.7717/peerj-cs.623.

[36] T. Shin. "Understanding Feature Importance in Machine Learning." https://builtin.com/data-science/feature-importance (accessed 19/04/2024.

[37] s.-o. contributors. "Bayesian optimization using Gaussian Processes." https://scikit-optimize.github.io/stable/modules/generated/skopt.gp_minimize.html (accessed 15/04/2024.

[38] scikit-learn. " Partial Dependence and Individual Conditional Expectation plots." https://scikit-learn.org/stable/modules/partial_dependence.html#:~:text=Partial%20dependence%20plots%20(PDP)%20show,the%20'complement'%20features). (accessed 09/04/2024.

[39] E. Berk. "Wind Turbine Scada Dataset." https://www.kaggle.com/datasets/berkerisen/wind-turbine-scada-dataset (accessed.

[40] U. Singh and M. Rizwan, "SCADA system dataset exploration and machine learning based forecast for wind turbines," *Results in Engineering,* vol. 16, p. 100640, 2022, doi: 10.1016/j.rineng.2022.100640.

[41] S. Media. "Aerial view of Wind Turbines in Yalova. 4K Footage in Turkey." https://www.shutterstock.com/video/clip-1100869339-aerial-view-wind-turbines-yalova-4k-footage?consentChanged=true (accessed 04/04/2024.

[42] B. Encyclopedia, "Faraday's laws of electrolysis | chemistry," in *Encyclopædia Britannica*, ed, 2016.

[43] F. Scheepers *et al.*, "Temperature optimization for improving polymer electrolyte membrane-water electrolysis system efficiency," *Applied Energy,* vol. 283, p. 116270, 2021, doi: 10.1016/j.apenergy.2020.116270.

[44] M. Chandesris, V. Médeau, N. Guillet, S. Chelghoum, D. Thoby, and F. Fouda-Onana, "Membrane degradation in PEM water electrolyzer: Numerical modeling and experimental evidence of the influence of temperature and current density," *International Journal of Hydrogen Energy,* vol. 40, no. 3, pp. 1353-1366, 2015, doi: 10.1016/j.ijhydene.2014.11.111.

[45] A. Yasper, D. Handoko, M. Putra, H. K. Aliwarga, and M. S. R. Rosid, "Hyperparameters Optimization in XGBoost Model for Rainfall Estimation: A Case Study in Pontianak City," *Jurnal Penelitian Pendidikan IPA,* vol. 9, no. 9, pp. 7113-7121, 2023, doi: 10.29303/jppipa.v9i9.3890.

[46] F. Maulana, Y. Engge, M. Nurhuda, and Istiroyah, "Effect of Voltage Source Differences on Hydrogen Gas Production by Electrolysis-photocatalysis Reactor Systems," *IOP Conference Series: Earth and Environmental Science,* vol. 1097, no. 1, p. 012049, 2022, doi: 10.1088/1755-1315/1097/1/012049.

[47] T. Fouda, S. Badr, A. Derbala, A. Elmetwalli, and A. Salamah, "Maximize hydrogen gas production from a small unit using acidic and saline water," *Scientific Papers Series Management, Economic Engineering in Agriculture & Rural Development,* vol. 18, no. 1, 2018.

[48] E. Amores, A. Contreras, L. Rodríguez, and M. Carrero, "Flow field optimization for PEM water electrolysis cell using computational fluid dynamic simulations (211)," 2018.

[49] M. Bonanno, K. Müller, B. Bensmann, R. Hanke-Rauschenbach, R. Peach, and S. Thiele, "Evaluation of the Efficiency of an Elevated Temperature Proton Exchange Membrane Water Electrolysis System," *Journal of The Electrochemical Society,* vol. 168, no. 9, p. 094504, 2021, doi: 10.1149/1945-7111/ac2188.

[50] S. Zhao *et al.*, "Calculating the Electrochemically Active Surface Area of Iridium Oxide in Operating Proton Exchange Membrane Electrolyzers," *Journal of The Electrochemical Society,* vol. 162, no. 12, pp. F1292-F1298, 2015, doi: 10.1149/2.0211512jes.

[51] F. Fouda-Onana, M. Chandesris, V. Médeau, S. Chelghoum, D. Thoby, and N. Guillet, "Investigation on the degradation of MEAs for PEM water electrolysers part I: Effects of testing conditions on MEA performances and membrane properties," *International Journal of Hydrogen Energy,* vol. 41, no. 38, pp. 16627-16636, 2016, doi: 10.1016/j.ijhydene.2016.07.125.

[52] B. Desalegn, D. Gebeyehu, B. Tamrat, T. Tadiwose, and A. Lata, "Onshore versus offshore wind power trends and recent study practices in modeling of wind turbines' life-cycle impact assessments," *Cleaner Engineering and Technology,* vol. 17, p. 100691, 2023, doi: 10.1016/j.clet.2023.100691.

8. Appendices

8.1 Appendix A: Hydrogen Production Simulation Graphs

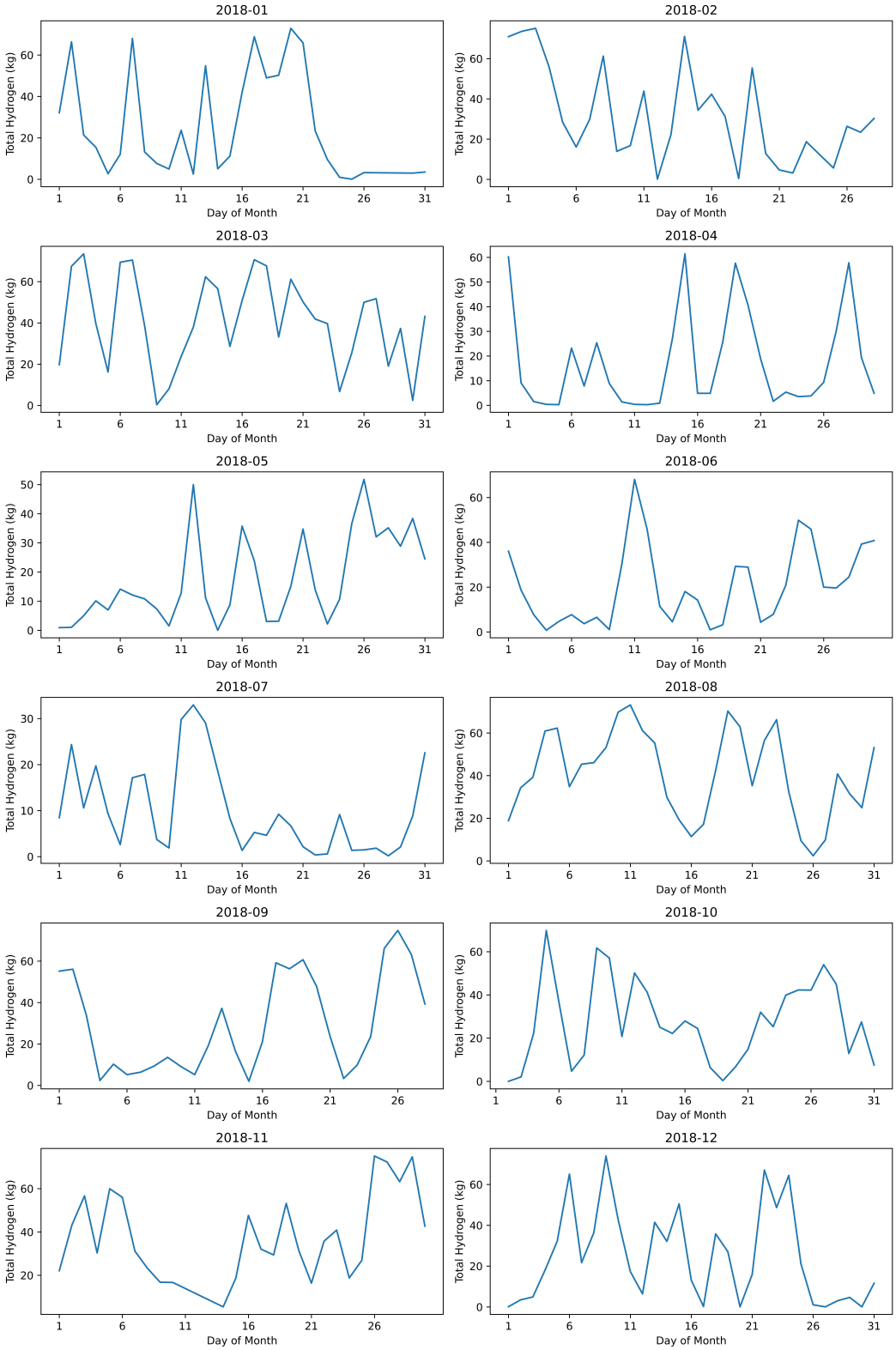


Figure 18 - Monthly Hydrogen production

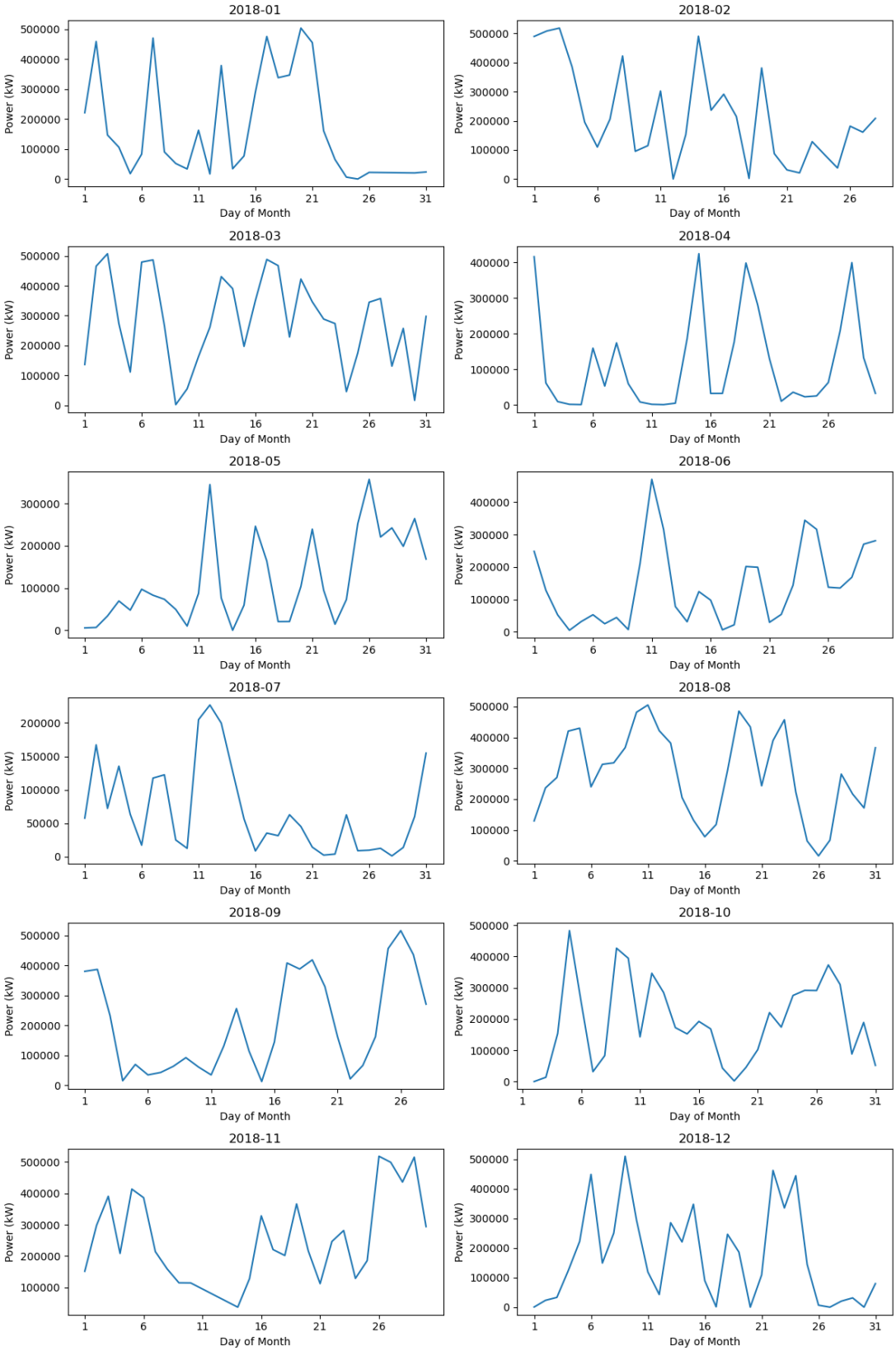


Figure 19 - Dynamic total power profile by month

8.2 Appendix B: Python Code

```
#import libraries:
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from scipy.stats import expon, reciprocal
from skopt.space import Real, Categorical, Integer
from skopt import gp_minimize
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from sklearn.inspection import permutation_importance
from sklearn.inspection import PartialDependenceDisplay
from sklearn.feature_selection import SelectFromModel
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np
from scipy.stats import randint, uniform, loguniform
import joblib
import xgboost as xgb

[ ]: #preprocessing steps:
pem = pd.read_csv("PEM_LARGE dataset.csv", encoding='ISO-8859-1')
# rename columns and set all to lowercase for consistency
pem.columns = pem.columns.str.lower().str.replace(' ', '_').str.replace('(', ␣
↪'').str.replace(')', '')
for col in pem.select_dtypes(include='object').columns: #assign category type ␣
↪to categorical features
        pem[col] = pem[col].astype('category')
#remove cell_design_type
pem.drop('cell_design_type', axis=1, inplace=True)
# Change flow_type to categorical as well
pem['flow_type_a_number'] = pem['flow_type_a_number'].astype('category')
pem['flow_type_c_number'] = pem['flow_type_c_number'].astype('category')
#drop these features
pem.drop('water_flow_rateml/min', inplace=True, axis=1)
pem.drop('cathode_flow_areacm2', inplace=True, axis=1)
right_skewed_columns = ['cell_voltage', 'cell_current_desa/cm2', ␣
↪'electrode_areacm2', 'anode_flow_areacm2', 'pressureatm']
for col in right_skewed_columns:
        # add 1 to all, to avoid log(0) error
        pem[col + '_log'] = np.log(pem[col] + 1)
# drop the original columns from the dataframe, so the log columns are the only ␣
```

```
↪ones that remain
pem.drop(right_skewed_columns, axis=1, inplace=True)
# remove power and powerw/cm2 from the pem dataframe
pem.drop(['powerw','powerw/cm2', 'electrolysis_type'], axis=1, inplace=True)
pem.head()

[ ]: #split and scale data
X = pem.drop(['hydrogen_flow_rate'] , axis=1) # Assign hydrogen flow rate as␣
↪target variable
Y = pem['hydrogen_flow_rate']
# initialize scaler
scaler = StandardScaler()
# cat and num columns
categorical_cols = pem.select_dtypes(include=['category']).columns
numerical_cols = X.select_dtypes(include=['float64', 'int64']).columns
#dummy coding
X=pd.get_dummies(X, categorical_cols)
#get PEM data
X[numerical_cols] = scaler.fit_transform(X[numerical_cols])
#split into train and test set
X_train_scaled, X_test_scaled, Y_train, Y_test = train_test_split(X, Y,␣
↪test_size=0.2, random_state=42)
X.head()
#create scaler mapping dataset
means = scaler.mean_
std_devs = scaler.scale_
scaling_info = pd.DataFrame({
        'Column': numerical_cols,
        'Mean': means,
        'Standard Deviation': std_devs
})
scaling_info.head()
# scaling_info.to_csv('Scaler_mean_mapping.csv')

[ ]: # functions
def performance_metrics(Y_test, y_pred):
        """Returns the performance metrics of a ml model"""
        mae = mean_absolute_error(Y_test, y_pred)
        mse = mean_squared_error(Y_test, y_pred)
        rmse = np.sqrt(mse)
        r2 = r2_score(Y_test, y_pred)
        print('R2 score:', r2)
        print('MSE:', mse)
        print('RMSE', rmse)
        print('MAE', mae)

def scaler_func(x, feature):
        """ scales a value for inputting into ML model based on its feature"""
        feature_row = mappings[mappings['Column'] == feature]
```

```
        if not feature_row.empty:
                mean = feature_row['Mean'].values[0]
                std_dev = feature_row['Standard Deviation'].values[0]
        # scale data
                scaled_data = (x - mean) / std_dev
                return scaled_data
        else:
                raise ValueError(f"Feature '{feature}' not found in scaler mappings.")

def scaler_func_log(x, feature):
        """ scales a value for inputting into ML model
        based on its log feature """
        x = np.log(x + 1)
        feature_row = mappings[mappings['Column'] == feature]
        if not feature_row.empty:
                mean = feature_row['Mean'].values[0]
                std_dev = feature_row['Standard Deviation'].values[0]
        # scale data
                scaled_data = (x - mean) / std_dev
                return scaled_data
        else:
                raise ValueError(f"Feature '{feature}' not found in scaler mappings.")
```

ML modelling

RF:

```
[ ]: # Hyperparamter tuning
# Define rf model
rf_model1 = RandomForestRegressor()
# Define hyperparameter parameters space
param_dist = {
'n_estimators': randint(100, 500),
'max_features': ['auto', 'sqrt', 'log2'],
'max_depth': randint(10, 100),
'min_samples_split': randint(2, 10),
'min_samples_leaf': randint(1, 4),
'bootstrap': [True, False]
}
# Set up random search with CV
random_search = RandomizedSearchCV(rf_model1, param_distributions=param_dist, ␣
↪n_iter=100, cv=5, verbose=2, random_state=42, n_jobs=-1)
random_search.fit(X_train_scaled, Y_train)
# best_parameters = random_search.best_params_
# print(best_parameters)

[ ]: # instantiate rf model with the tuned hyperparameters
rf_model_tuned = RandomForestRegressor(n_estimators=322, max_depth=76, ␣
↪max_features='sqrt', min_samples_leaf=1, min_samples_split=3, ␣
↪bootstrap=False)
#fit the model
```

```
rf_model_tuned.fit(X_train_scaled, Y_train)
#use rf to find most important features
feature_importance_rf = rf_model_tuned.feature_importances_
# joblib.dump(rf_model_tuned, 'rf_tuned.joblib') # save ML model
y_pred = rf_model_tuned.predict(X_test_scaled)
```

DTR:

```
[ ]: #Hyperparameter tuning:
# Define dtr model
dt_model = DecisionTreeRegressor()
# Define hyperparameter parameters space
param_dist = {
'max_depth': [None] + list(range(1, 20)),
'min_samples_split': randint(2, 20),
'min_samples_leaf': randint(1, 20),
'max_features': ['auto', 'sqrt', 'log2', None]
}
# Set up the random search with CV
random_search_dt = RandomizedSearchCV(dt_model, param_distributions=param_dist,␣
↪n_iter=100, cv=5, verbose=2, random_state=42, n_jobs=-1)
random_search_dt.fit(X_train_scaled, Y_train)
# #print best params for dt model
# best_params_dt = random_search_dt.best_params_
# print(best_params_dt)
```

```
[ ]: # instantiate dt model with tuned hyperparamaters
dt_model_tuned = DecisionTreeRegressor(max_depth=11, max_features='log2',␣
↪min_samples_leaf=1, min_samples_split=2)
#fit model
dt_model_tuned.fit(X_train_scaled, Y_train)
#
y_pred_dt = dt_model_tuned.predict(X_test_scaled)
# use dt to get feature importance

feature_importance_dt = dt_model_tuned.feature_importances_
```

XGB:

```
[ ]: # XGB
xgb_regressor1 = xgb.XGBRegressor()
params = {
'n_estimators': randint(100, 501),
'learning_rate': uniform(0.01, 0.3),
'max_depth': randint(1, 9),
'colsample_bytree': uniform(0.3, 0.4),
'min_child_weight': randint(1, 6),
'subsample': uniform(0.5, 0.4)
}
random_search = RandomizedSearchCV(xgb_regressor1, param_distributions=params,␣
↪n_iter=100, n_jobs=-1, cv=5, verbose=2, random_state=42)
```

```
random_search.fit(X_train, Y_train)
best_model = random_search.best_estimator_
print("Best hyperparameters:", random_search.best_params_)

[ ]: #instantiate xgb tuned
xgb_regressor_tuned = xgb.XGBRegressor(subsample=0.9, n_estimators=400, ␣
↪min_child_weight=1, max_depth=3, learning_rate=0.05, colsample_bytree=0.7)
#fit model
xgb_regressor_tuned.fit(X_train_scaled ,Y_train)
#get performance metrics
y_pred_xgb = xgb_regressor_tuned.predict(X_test_scaled)
# use dt to plot feature importance
feature_importance_xgb = xgb_regressor_tuned.feature_importances_
```

KNN:

```
[ ]: #instantiate model
knn_regressor = KNeighborsRegressor()
#Set up hyperparameter space
params = {
'n_neighbors': np.arange(1, 31),
'weights': ['uniform', 'distance'],
'algorithm': ['ball_tree', 'kd_tree', 'brute', 'auto'],
'leaf_size': np.arange(1, 50),
'p': [1, 2],
'metric': ['euclidean', 'manhattan', 'chebyshev', 'minkowski']
}
random_search_knn = RandomizedSearchCV(knn_regressor, ␣
↪param_distributions=params, n_iter=100, n_jobs=-1, cv=5, verbose=3, ␣
↪random_state=42)

random_search_knn.fit(X_train_scaled, Y_train)
# Best model and hyperparameters
best_model_knn = random_search_knn.best_estimator_
print("Best hyperparameters for KNN Regression:", random_search_knn.
↪best_params_)
[ ]: # instantiate knn regressor tuned

knn_regressor_tuned = KNeighborsRegressor(weights='distance', p=1, ␣
↪n_neighbors=2, metric='manhattan', leaf_size=38, algorithm='brute')
#fit model
knn_regressor_tuned.fit(X_train_scaled, Y_train)
y_pred_knn = knn_regressor_tuned.predict(X_test_scaled)
# for knn can use permutation importance to get featuer importance
result = permutation_importance(knn_regressor_tuned, X_test_scaled.toarray(), ␣
↪Y_test, n_repeats=10, random_state=42, n_jobs=-1)

[ ]: # graphs code
# for prediction accuracy plots
plt.figure(figsize=(7, 5))
```

```
plt.scatter(Y_test, y_pred, color='blue')
plt.plot([y_pred.min(), y_pred.max()], [y_pred.min(), y_pred.max()], 'r--', lw=2)
plt.xlabel('Actual H', fontweight='bold', fontsize=15)
plt.ylabel('Predicted H', fontweight='bold', fontsize=15)
plt.title('Actual vs Predicted')
# annotate performance metrics
plt.text(0.70, 0.5, f'R2 = {R2:.3f}\nRMSE = {RMSE:.3f}\nMAE = {MAE:.3f}', fontsize=14, fontweight='bold',
verticalalignment='top', horizontalalignment='left',
transform=plt.gca().transAxes,
bbox=dict(facecolor='white', alpha=0.5))
plt.text(0.01,0.96, '(C)', fontsize=15, color='red',
verticalalignment='center', horizontalalignment='left',
transform=plt.gca().transAxes,
)
plt.xticks(fontsize=14, fontweight='bold')
plt.yticks(fontsize=14, fontweight='bold')
plt.tight_layout() # tight layout
# plt.savefig('prediction_accuracy_plot.png', dpi=1000)
plt.show()
```

```
[ ]: # for feature imoprtance plots
#plot relative importance of the features
indices = np.argsort(feature_importance)[::-1]
#plot top N most important features
N = 10
plt.figure(figsize=(10, 6))
plt.title("Top 10 Feature Importances")
plt.bar(range(N), feature_importance[indices[:N]], align="center")
plt.xticks(range(N), X.columns[indices[:N]], rotation=90)
plt.xlim([-1, N])
plt.show()
```

Optimisation

```
[ ]: continuous_params_list = ['cell_design_number', 'temprature_cellk', 'cell_voltage_log', 'cell_current_desa/cm2_log',
'electrode_areacm2_log', 'anode_flow_areacm2_log', 'pressueratm_log']
space = [
Real(1, 10, name='cell_design_number'),
Real(293.15, 403.15, name='temprature_cellk'),
Real(0.867100, 2.995732, name='cell_voltage_log'),
Real(0.0002408010051107634, 1.0986122886681098, name ='cell_current_desa/cm2_log'), # parameter space
Real(2.302585092994046, 4.61512051684126, name='electrode_areacm2_log'),
Real(1.6094379124341003, 3.9318256327243257, name='anode_flow_areacm2_log'),
Real(0.6931471805599453, 1.3862943611198906, name='pressueratm_log'),
```

```
]

[ ]: def objective(params):
        cell_design_number, temprature_cellk, cell_voltage_log, ␣
↪cell_current_desacm2_log, electrode_areacm2_log,
        anode_flow_areacm2_log, pressueratm_log = params
        input_data = pd.DataFrame({'cell_design_number': [cell_design_number],
        'temperature_cellk': [temperature_cellk],
        'cell_voltage_log': [cell_voltage_log], # change ␣
↪to df for input into ml model
        'cell_current_desa/cm2_log': ␣
↪[cell_current_desacm2_log],
        'electrode_areacm2_log': [electrode_areacm2_log],
        'anode_flow_areacm2_log': ␣
↪[anode_flow_areacm2_log],
        'pressueratm_log': [pressueratm_log]})
        scaled_input = scaler.transform(input_data)
        prediction = rf_model_tuned.predict(scaled_input)

        return -prediction[0] # - sign as minimise function, going to maximise ␣
↪output from RF

[ ]: # Perform bayesian optimisation
result = gp_minimize(objective, space, n_calls=50, random_state=42)
# print best parameters
print("Best parameters: {}".format(result.x))

[ ]: # check optimal configuration
row = X.iloc[1:2]
dummy_df = X.select_dtypes(include='bool')
dummy_list = dummy_df.columns.tolist()
print(dummy_list)
for dummy in dummy_list:
        row[dummy] = False
# set up the optimal configuration
row['anode_type_Titanium'] = True
row['cathode_type_Titanium'] = True
row['anode_gde_titanium_mesh'] = True
row['cathode_gde_porous_titanium'] = True
row['membrane_type_Nafion_110'] = True
row['cathode_catalyst_0.4_mg_Pt/cm2'] = True
row['anode_catalyst_2_mg_Ir/cm2+2_mg_pt/cm2'] = True
row['catholyte_DI_Water'] = True
row['anolyte_DI_Water'] = True
row['electrode_shape_round'] = True
row['flow_type_a_number_29'] = True
row['flow_type_c_number_29'] = True
# numerical parameters:
row['cell_design_number'] = scaler_func(10, 'cell_design_number')
```

```
row['temprature_cellk'] = scaler_func(337.03, 'temprature_cellk')
row['cell_current_desa/cm2_log'] = scaler_func_log(2, 'cell_current_desa/
↪cm2_log')
row['electrode_areacm2_log'] = scaler_func_log(80.86, 'electrode_areacm2_log')
row['anode_flow_areacm2_log'] = scaler_func_log(50, 'anode_flow_areacm2_log')
row['pressueratm_log'] = scaler_func_log(1, 'pressueratm_log')
row['cell_voltage_log'] = scaler_func_log(19, 'cell_voltage_log')
row.head()
```

```
[ ]: # hydrogen flow rate with optimal configuration
prediction = rf_model_tuned.predict(row)
print(prediction)
```

```
[ ]: # partial dependence plots
features = [0,1, 2, 3, 4, 5, 6] # e.g temp is 1
#for example temperature code
features = [1]
fig, ax = plt.subplots(figsize=(8, 5))
# Create the partial dependence plot
display = PartialDependenceDisplay.from_estimator(rf_model_tuned, X, features, ␣
↪ax=ax)
#change labels
original_ticks = np.linspace(-1.5, 2, num=8)
new_labels = np.linspace(300, 400, num=8).astype(int)
plt.yticks(fontweight='bold', fontsize=14)
plt.xticks(ticks=original_ticks, labels=new_labels, fontweight='bold', ␣
↪fontsize=14)
plt.xlabel('Temperature (K)', fontweight='bold', fontsize=14)
plt.ylabel('Hydrogen flow rate (ml/min)', fontweight='bold', fontsize=14)
plt.tight_layout()
plt.savefig('temp partial dependence.png', dpi=500)
plt.show()
```

Hydrogen production sim:

```
[ ]: def calculate_voltage_current(wattage, base_voltage=19, base_current=100):
        ratio = base_voltage / base_current # ratio of voltage:current
        current = (wattage / (ratio**2))**0.5 # power split into current
        voltage = ratio * current # power split into voltage
        return voltage, current
```

```
[ ]: # setup input into model
row_change = row.copy()
row_change.head()
row_change_1 = row.copy()
```

```
[ ]: # import wind power dataset
wind_df = pd.read_csv("T1.csv")
wind_df.head()
new_df = wind_df[['Date/Time', 'LV ActivePower (kW)']].copy()
```

```
#remove all negative values
new_df = new_df[new_df["LV ActivePower (kW)"] >= 0]
new_df.reset_index(drop=True, inplace=True)
new_df.head()

[ ]: # simulation:
hydrogen_result = []
for i in new_df['LV ActivePower (kW)']:
        power = i * 10 ** 3
        if power < 19000:
                power = power / 10 # 10 cell electrolyser
                voltage, current = calculate_voltage_current(power)
                current_dens = current / 50 # current divided by area to get current␣
↪density
                voltage_log_remainder = scaler_func_log(voltage, 'cell_voltage_log')
                current_dens_log_remainder = scaler_func_log(current_dens,␣
↪'cell_current_desa/cm2_log')
                row_change_1['cell_voltage_log'] = voltage_log_remainder
                row_change_1['cell_current_desa/cm2_log'] = current_dens_log_remainder
                hydrogen = rf_model_tuned.predict(row_change_1)
                hydrogen = hydrogen[0] * 10 # as every 10 minutes
                hydrogen_result.append(hydrogen)
        else:
                quotient, remainder = divmod(power, 19000)
                hydrogen = rf_model_tuned.predict(row)
                hydrogen = hydrogen[0] * 10 * quotient # as reading every 10 minutes␣
↪and quotient will be the number of electrolysers at full power
                voltage, current = calculate_voltage_current(remainder)
                current_dens = current / 50
                voltage_log_remainder = scaler_func_log(voltage, 'cell_voltage_log')
                current_dens_log_remainder = scaler_func_log(current_dens,␣
↪'cell_current_desa/cm2_log')
                row_change['cell_voltage_log'] = voltage_log_remainder
                row_change['cell_current_desa/cm2_log'] = current_dens_log_remainder
                hydrogen_1 = rf_model_tuned.predict(row_change)
                hydrogen_1 = hydrogen_1[0] * 10 # as reading every 10 minutes
                hydrogen_result.append(hydrogen + hydrogen_1) # total hydrogen produced␣
↪in ml
df = pd.DataFrame(hydrogen_result, columns=['Hydrogen ml'])
# df.to_csv('hydrogen_result_1.csv')


[ ]: new_df['Date/Time'] = pd.to_datetime(new_df['Date/Time'], format='%d %m %Y %H:
↪%M') # new_df to datetime

[ ]: # for power profile
feb_fourth_power = new_df[new_df['Date/Time'].dt.date == pd.
↪to_datetime('2018-02-04').date()]
plt.figure(figsize=(9, 5))
```

```
graph = plt.plot(feb_fourth_power['Date/Time'], feb_fourth_power['LV␣
↪ActivePower (kW)'], marker='o', markersize=4)
plt.title('Wind power profile')
plt.xlabel('Time', fontweight='bold', fontsize=11)
plt.ylabel('Power (kW)', fontweight='bold', fontsize=11)
plt.grid(True)
# format x axis
hours_fmt = mdates.DateFormatter('%H:%M')
plt.gca().xaxis.set_major_formatter(hours_fmt)
plt.gca().xaxis.set_major_locator(mdates.HourLocator(interval=3))
# x axis lims
plt.xlim(feb_fourth_power['Date/Time'].min(), feb_fourth_power['Date/Time'].
↪max())
plt.xticks(fontweight='bold', fontsize=11)
plt.yticks(fontweight='bold', fontsize=11)
plt.tight_layout()
plt.show()
```

```
[ ]: # sort out hydrogen ml to kg conversion
hydrogen_result_df = pd.read_csv('hydrogen_result.csv')
hydrogen_result_df.head()
hydrogen_result_df['Hydrogen ml'] = hydrogen_result_df['Hydrogen ml'] * 8.2e-8
hydrogen_result_df = hydrogen_result_df.rename(columns={'Hydrogen ml':␣
↪'Hydrogen (kg)'})
hydrogen_result_df.head()
#combine the dataset with the new_df dataset
new_df['Hydrogen (kg)'] = hydrogen_result_df['Hydrogen (kg)']
new_df.head()
# create a new data frame called days_df, and combine the hydrogen production␣
↪data based on days
days_df = new_df
# days_df.to_csv('days_df.csv')
# Convert date time to actualt date time format

days_df['Date/Time'] = pd.to_datetime(days_df['Date/Time'], format='%d %m %Y %H:
↪%M')
# sum hydrogen kg column and group this summation by day
daily_hydrogen = days_df.groupby(days_df['Date/Time'].dt.date)['Hydrogen (kg)'].
↪sum().reset_index(name='Total Hydrogen (kg)')
daily_hydrogen.head()
# daily_hydrogen.to_csv('daily_hydrogen.csv')
```

```
[ ]: # for hydrogen production on feb fourth,t
feb_fourth = days_df[days_df['Date/Time'].dt.date == pd.
↪to_datetime('2018-02-04').date()]
plt.figure(figsize=(9, 5))
graph = plt.plot(feb_fourth['Date/Time'], feb_fourth['Hydrogen (kg)'],␣
↪marker='o', markersize=4)
plt.title('Hydrogen Production on February 4th, 2018')
```

```
plt.xlabel('Time', fontweight='bold', fontsize=11)
plt.ylabel('Hydrogen (kg)', fontweight='bold', fontsize=11)
plt.grid(True)
# format x axis
hours_fmt = mdates.DateFormatter('%H:%M')
plt.gca().xaxis.set_major_formatter(hours_fmt)
plt.gca().xaxis.set_major_locator(mdates.HourLocator(interval=3))
# x axis lims
plt.xlim(feb_fourth['Date/Time'].min(), feb_fourth['Date/Time'].max())
plt.xticks(fontweight='bold', fontsize=11)
plt.yticks(fontweight='bold', fontsize=11)
plt.tight_layout()
plt.show()
```

```
[ ]: # resample and sum hydrogen production by month
monthly_hydrogen_production_1 = daily_hydrogen['Total Hydrogen (kg)'].
↪resample('M').sum()
monthly_production_1 = pd.DataFrame(monthly_hydrogen_production_1)
new_labels = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'June', 'Jul', 'Aug', 'Sep',␣
↪'Oct', 'Nov', 'Dec']
monthly_production_1['Months'] = new_labels
monthly_production_1.head()
```

```
[ ]: # for hydrogen every month
new_labels = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'June', 'Jul', 'Aug', 'Sep',␣
↪'Oct', 'Nov', 'Dec']
plt.figure(figsize=(8, 5))
plt.plot(monthly_production_1['Months'], monthly_production_1['Total Hydrogen␣
↪(kg)'] , marker='o')
plt.title('Total Hydrogen Produced Each Month')
plt.xlabel('Month', fontweight='bold', fontsize=11)
plt.ylabel('Hydrogen (kg)', fontweight='bold', fontsize=11)
plt.xticks(fontweight='bold', fontsize=11)
plt.yticks(fontweight='bold', fontsize=11)
plt.grid(True)
plt.tight_layout()
plt.show()
```